SAPIENZA
UNIVERSITÀ DI ROMA

CereMath: a library of ML kernels for dataflow architectures

Department of Computer Science
Applied Computer Science and Artificial Intelligence

Leonardo Biason
ID number 2045751

Advisor
Prof. De Sensi Daniele

Academic Year 2025/2026

Thesis defended on March 27th, 2026
in front of a Board of Examiners composed by:

Prof. Tolomei Gabriele (chairman)

Prof. De Sensi Daniele

Prof. Maselli Gaia

CereMath: a library of ML kernels for dataflow architectures
BSc thesis. Sapienza University of Rome

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: leonardo@biason.org

To who made this journey special
May this courage lead forward

Abstract

In modern days, most computers are built by following the von Neumann architecture, with the reason being the efficiency of said general purpose model. However, its greatest bottleneck is data transfer: indeed, at high bandwidths, this architectural paradigm struggles to keep an elevated throughput. Domain-specific architectures have been arising and re-emerging, especially with the increasing interest in AI and HPC applications: one such model is the dataflow architecture. Cerebras, a US-based company, designed a so-(affectionately)-called "*dinner-plate*" sized chip based on said paradigm, called the Wafer Scale Engine (WSE), which aims to ensure parallelism by emphasizing the data transfer within the chip. This thesis will introduce a library of mathematical kernels, adapted for the WSE, alongside an illustration of how to connect said kernels to achieve efficiency. Afterward, a comparison with CPU and GPU versions will be shown. Finally, there will be a concise discussion about whether such architecture may be suitable for the field that this work analyzed, and whether it may see a future ahead in this constantly evolving world for also other scenarios and fields, such as HPC.

Oggigiorno la maggior parte dei computer è costruita seguendo l'architettura di Von Neumann, principalmente grazie all'efficienza di questo modello generico e versatile. Tuttavia, in questo paradigma lo spostamento dei dati rappresenta il collo di bottiglia più importante: infatti, quanto più la quantità di dati richiesta aumenta, tanto più l'architettura fallisce nello scalare delle performance. Per questo motivo si è vista negli scorsi anni la nascita e la ri-scoperta di architetture per domini specifici, specialmente grazie al maggiore interesse nei campi dell'IA e dell'HPC: una di queste architetture è l'architettura dataflow. Cerebras, una società statunitense, ha creato un chip dalle dimensioni di un wafer, basato su tale architettura dataflow; questo chip è il Wafer Scale Engine (WSE), che punta all'ottenere parallelismo e buone performance attraverso lo spostamento dei dati. In questa tesi verrà introdotta una libreria di kernel e funzioni matematiche, adattate per il WSE, e verrà inoltre mostrato come connettere queste funzioni per dimostrarne l'efficienza. Seguirà un confronto con delle versioni per CPU e GPU. La tesi si concluderà con una riflessione sulla validità di tali architetture, e se queste abbiano o meno un futuro in campi diversi da quelli per cui sono state pensate, come quello dell'HPC.

Contents

Chapter 1
Towards a dataflow architecture

Since the birth of early computers and the release of the First Draft of a Report on the EDVAC [30], computers have always been developed by having a specific architecture in mind, called the **Von Neumann architecture** (in Figure 1.1). For such architecture, there are four main components, which are able to communicate through interconnects of different kinds and bandwidths. Said components are the following:

- the **Central Processing Unit** (CPU), which is capable of performing logic and arithmetical operations, and that contains elements such as the Control Unit (which determines the operations to be carried out at a given moment), the Arithmetic Logic Unit (also denoted as ALU, is the element responsible for performing a given logic or arithmetic operation), and a set of Registers for quick data retention;
- the **Memory Unit**, whose task is to store all the data needed from the CPU in order to execute its tasks. The kind of data that could be memorized ranges from the instructions of a program to the data that needs to be processed;
- several **Input** and **Output devices** (I/O), which allow external agents or peripherals to interact with the computer.

The reason why this architecture still survives today is because of its general-purpose nature, which, in a time of constant evolution in the computer science field such as the early 50's, revealed itself to be rather convenient. Today, we still take advantage of the Von Neumann architecture for many kinds of computers: from the ones that may be sitting on our desks to the ones housed in computing centers, and all of this is because of the proven effectiveness of said architecture.

However, this architecture has its downsides, which are mostly given by the need of constantly transferring data and instructions from the main memory to the registers;

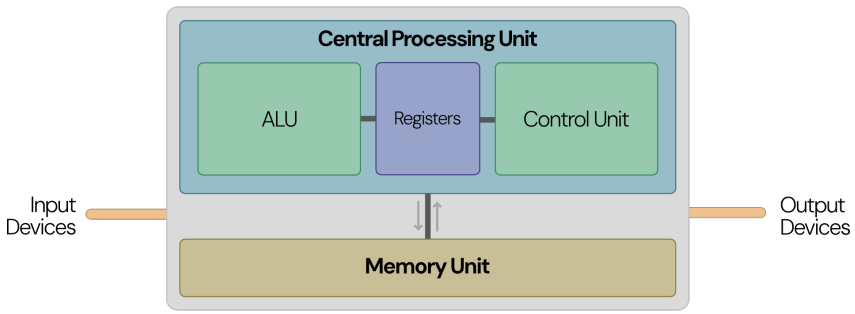

Figure 1.1. Overview of the Von Neumann architecture

given that the interconnects have a limited bandwidth, computations that heavily rely on huge amount of data are significantly slowed down. This problem is also called the "*Von Neumann bottleneck*". While this does not represent a major issue in consumer electronics, it actually impacts high-performance jobs and tasks, such as AI training and inference.

Before settling on this paradigm we saw a lot of experimentation, especially in the early stages of computer science, for understanding the effect of different architectures. A specific kind of architecture started being researched at the MIT during 1976 [3], and this was the **dataflow architecture**. The core idea of such architecture was not to move data and program to the computing elements, but rather to just move the data and process it as it flows through the circuit; an idea that is deeply in contrast with the Von Neumann paradigm. The research conducted at MIT failed to give enough reasons to replace the Von Neumann architecture, which led to the closure of the dedicated research group in 1995.

Although dataflow concepts have been employed in some computer science fields (one of many, in networking), until the last few years it remained confined to niche areas. Recently however, we have seen the industry populating with new, experimental hardware, mostly to take advantage of the AI phenomenon. It's in this period that also the dataflow architecture re-emerged, both in HPC settings, mostly thanks to the effort of companies such as Cerebras and Sambanova, and in embedded platforms, with the Electron E1 microprocessor from Efficient [2]. Before delving into the analysis of the products proposed from one of the aforementioned companies, Cerebras, it's important to first discuss the setting that moved the research, development and re-discovery of this architecture: the **explosion of the AI market** and the rise of LLM models.

1.1 The AI bubble and the rise of Cerebras

Since the rise of generative language models such as GPT-3, AI has been on everyone's lips: businesses have seen an opportunity in this category of algorithms, following the extreme adoption of these models in people's everyday lives. As a consequence, many companies started developing interest in having their own versions of these tools, requiring thus powerful hardware for training and performing inference of their models.

It's especially in this period that the limitations of the Von Neumann architecture have been seen: training an AI model is an intensive data-driven task, because of the need of storing huge amounts of parameters (for scale, models such as Grok 4 are estimated to have 3.0×10^{12} parameters!) and the need to use an ever-growing amount of training data. Training a model is not a task suitable for highly sequential hardware such as CPUs: the computations that must be done are many and repetitive, and even by using together CPUs with multiple cores we would most likely incur communication and/or data transfer overhead. Graphics Processing Units (GPUs) allow to partly overcome these issues: these are a particular kind of hardware designed to perform computations on high amounts of data, in a parallel fashion.

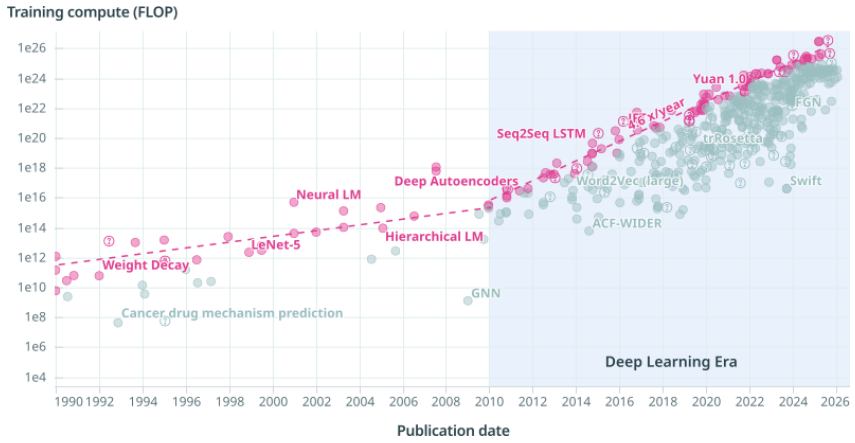

Figure 1.2. Released ML/DL models from 1990 to 2026 against the FLOPs needed for their training. Courtesy of epoch.ai [1]

While originally GPUs were meant only for graphics purposes (hence the name), companies such as NVIDIA and AMD have been focusing on creating General Purpose-GPUs (GPGPUs), with particular additions to the chip of these accelerators. An example is the introduction, starting from the Volta generation, of **Tensor Cores** on NVIDIA GPUs: these are dedicated cores that perform only a single operation, being a general matrix-matrix multiplication and accumulation ($\mathbf{C} = \alpha\mathbf{A} \times \mathbf{B} + \beta\mathbf{C}$, called GEMM by the BLAS standard [21]) in a single clock cycle. Given the trend of recent models (Figure 1.2), the amount of compute power continues to grow with each new model, hence making crucial any kind of hardware that can speedup important operations such as the GEMM.

It is not by chance that hardware support was added for speeding up matrix-matrix operations: in fact, AI algorithms heavily rely on such operations, and thus having specialized and dedicated hardware seemed the key to speed up AI-related tasks. It's for this reason that companies started working on their domain-specific architectures (DSA), such as Google with their Tensor Processing Units (TPUs) [15]. Among the many kinds of different architectures that were being developed, a company in particular decided to focus on the now-"dead" dataflow architecture: **Cerebras**.

The Cerebras company was founded in 2015 with the idea of creating a wafer-scaled chip that would surpass currently used hardware accelerators in terms of computing power, and released in 2019 their first iteration of their (affectionally called) "*dinner-plate sized*" chip: the **Wafer Scale Engine** (WSE), which can be seen in Figure 1.3. From then on, they have released other 2 generations of the chip alongside its housing, the Cerebras System (CS). The idea was promising: obtaining performance by making data move through a constantly-operating chip, analogously to an assembly line.

While being an experimental architecture, Cerebras allows to reach high performances (namely, 125 PetaFLOP/s of FP16 operations [10, 9] per WSE, and 256 ExaFLOP/s [7] in clusters of 2048 WSEs) with an intuitive Python interface for AI-related jobs. However, it also allows to write custom programs through a combination of Python

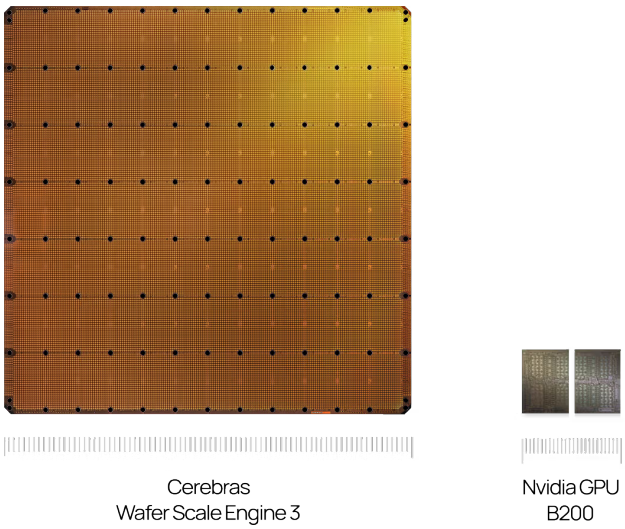

Figure 1.3. A Cerebras WSE-3 shown in comparison with the die of an NVIDIA Blackwell GPU. Courtesy of Cerebras

and Cerebras System Language (CSL), a custom proprietary language made for writing kernels that can be executed directly on the WSE.

Given the novelty of such architecture, it's still hard to find details on the inner workings (both on the hardware and software side) of the WSE, despite the vast interest by researchers on developing tailor-suited models for this architecture [19]. In fact, while bindings for common AI frameworks such as PyTorch and Tensorflow are available to developers, it's not clear how the called operations are executed on the WSE: how are the operations placed on the WSE, which kind of instructions are used, etc... For this reason, it's unknown whether the basic operations used by the WSE for AI tasks are optimal or not, and whether further optimization could be done.

In this thesis, moved by this uncertainty, we want to explore the possibility of reaching higher performances through the writing of custom mathematical kernels, which are commonly seen in the Machine Learning (ML) and Deep Learning (DL) settings. Such operations will be the Convolution, the Pooling (both Max and Average versions), the General Matrix-Vector operation (GEMV) and the General Matrix-Matrix operation (GEMM). The set of these operations will be called and referred to as "**CereMath**".

This work will thus see the following sections:

- chapter 2 will provide a description of the Cerebras WSE-3, their latest generation of the Wafer Scale Engine, and the introduction of the setting on which this work has been conducted;

- chapter 3 will give an overview of the state-of-the-art (SotA) of the ML kernels that have been analyzed, and a thorough description of how they have been implemented for the Cerebras WSE;

- chapter 4 will present an evaluation of the work previously described, comparing the produced versions to the SotA solutions that are already available;
- chapter 5 will finally summarize the work done, with final considerations on the future of this architecture and possible expansions of this work.

Chapter 2
The Cerebras Wafer Scale Engine 3

Since 2019, Cerebras has released 3 generations of their WSE, with the last one, the 3rd, being released in 2024. Throughout this work, we will consider as target architecture the WSE-3, being the most recent and most available in modern datacenters.

The WSE-3 contains 4 trillion transistors in a total area of $4.6225 \cdot 10^4$ mm^2 [31]: for scale, an NVIDIA H100 has a die surface of 814 mm^2 (Figure 1.3 in the previous chapter showed a surface comparison of the WSE-3 against an NVIDIA Blackwell GPU die). The WSE-3 chip counts 900,000 "working" cores (or Processing Elements, PEs), all manufactured with a 5nm TSMC process. We specify "working" because of the yielding problem: it's not uncommon when producing multicore chips that some cores may be defective, and Cerebras is no stranger to this. However, the company guarantees that 900,000 of the 970,000 physical PEs will work for each produced WSE-3 [31] (hence retaining $\simeq 93\%$ of the manufactured cores).

To produce a WSE, Cerebras prints onto a silicon wafer singular blocks of PEs, internally called "dies", which are then connected by printing the internal interconnects. A die contains $\simeq$ 10000 working PEs, and each WSE-3 contains around 90 dies, displaced in a 2D mesh. [23]

By itself the WSE-3 is not capable of interfacing with any external device, hence the reason why Cerebras ships it within a Cerebras System 3 (CS-3): this is an independent system which is capable of providing not only I/O capabilities to the WSE-3, but also cooling, power and networking. Multiple CS-3 systems can be arranged into a cluster, up to 2048 machines connected together. This way not only the WSE-3 provides scaling-up opportunities, but also, up to a certain extent, scale-out possibilities, allowing to reach even higher performances.

In order to connect with other systems, each CS-3 is equipped with a 12× 100 Gbps Ethernet ports switch. The size of a CS-3 moreover allows to place from 2 to more (upon client request) into a single rack, easing the setup of an independent cluster. For the scope of this work, however, we will only use a singular WSE-3 for the evaluation of the experiments.

2.1 Architecture of the WSE-3

The reason why the WSE-3 is considered a dataflow architecture is because of the topology of its cores and the way they can connect to each other. The 900,000 PEs are placed in a 2-dimensional grid, forming a rectangle of 762×1176 cores. Each PE is independent with respect to the other cores, having its own Program Counter (PC) and SRAM. The PEs also house each a fabric router, which is a special hardware circuit that allows to programmatically manage the routing of data from/to the PE.

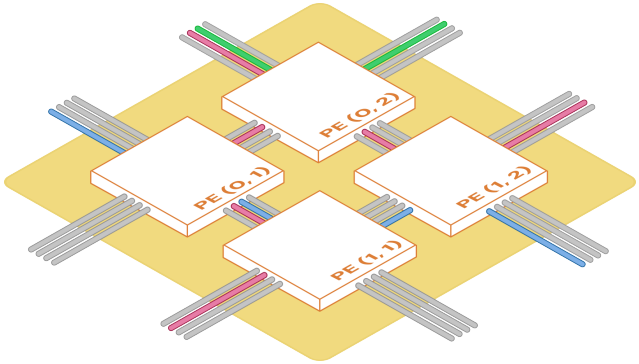

Figure 2.1. Isometric view of a sample 2×2 grid of PEs

An example of a small mesh of PEs can be seen in Figure 2.1. Within a WSE, data travels in packets of 32 bits, and such packets are called **wavelets**. A better overview of how wavelets are sent and can be used is explained in a later part of this work.

In the following sections an explanation of the WSE hardware and programming model will follow. It's important however to note that most of the information here reported regarding the hardware comes either from the internal documentation of the Cerebras SDK [8] or from posts in the Cerebras Developers forum, where official employees from the company reply and interact. This is mostly due to the closed-source nature of the SDK and the novelty of the architecture. Hence, not all the information can be independently confirmed, and a certain degree of trust must be used when dealing with some details of the WSE.

2.1.1 The Processing Element (PE)

A Processing Element (PE) is a $3.8 \cdot 10^4$ μm^2-sized core, and is composed of the following 3 elements: its Compute Engine (CE), which executes the instructions of the program at a frequency of 875 MHz; a local SRAM memory of 48 kB, which is private for each PE; a fabric router, which can be programmed by the user and the PE, and allows different neighbouring PEs to communicate between each other. The positioning of such elements on the PE's area can be observed in Figure 2.2.

The memory in the WSE is structured in 8 banks of 6 kB each, and in certain conditions the WSE can execute some instructions in Single-Instruction Multiple-Data (SIMD) mode[1], performing two read and one write operations of 32 bits at the same clock cycle.

Because each PE is independent with respect to the flow execution of the WSE, this means that all PEs must store independently their program. The WSE doesn't adopt a shared-memory approach for the various cores, so all the instructions and data that a PE will need must be stored within the PEs SRAM. This is much different from GPGPUs for instance, where the code is shared between threads in the same warp. However, this places an important boundary on the amount of data that each PE can store, making data distribution a key element of the WSE programming model.

[1]Not all operations allow for SIMD mode, and the size of vectors changes between generations of WSE. The ones available can be found at https://sdk.cerebras.net/csl/language/appendix

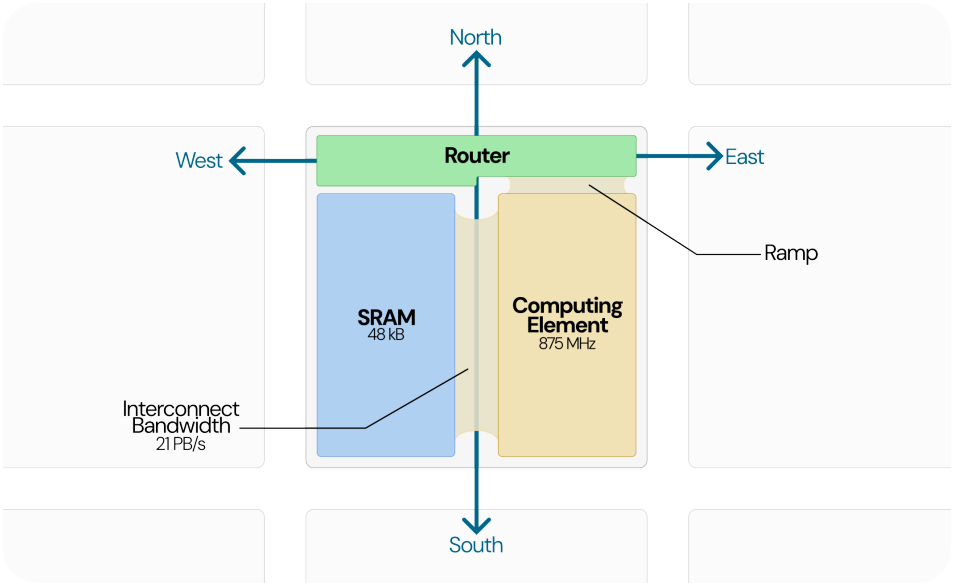

Figure 2.2. Conceptual overview of a PE and its internal components

Another key difference between the WSE and modern memory architectures is the absence of a cache. While this may seem as a downside with respect to the Von Neumann architecture, it's actually a design choice from Cerebras: for the WSE, the high fabric bandwidth of 21 PB/s [23] removes the need of complex memory hierarchies and the overhead introduced by a cache system. The high bandwidth is achieved by making so that the CE and the SRAM equally share the area of the PE, making the data travel for short distances quickly. There is a simple cache in the WSE, but it's dedicated to store data structure information. This will be further discussed together with data structures later on.

The fabric router of the PE is what allows the WSE to have dataflow capabilities, thanks to 5 hardware routes that can be independently activated (both at compile and runtime):

- the `NORTH`, `SOUTH`, `EAST` and `WEST` routes are meant for communicating with other PEs;
- the `RAMP` route connects the router with the CE.

Since the intra-PE communication happens between the router and the CE, it's thus impossible for external PEs to access another PE's internal SRAM. The router is however responsible for holding data in both input and output queues until the CE is free to process the incoming data (or until the next router in the communication chain is free to accept new data). Such queues are independent from the SRAM.

Routes between PEs can be defined programmatically through colors, which are virtual communication channels. More information about their working will be given later on.

2.2 The programming model

In order to execute code on the WSE, there are two available ways: the programmer can either choose to use the official Cerebras bindings for frameworks such as PyTorch or Tensorflow, or it can write its own code in the Cerebras Software Language (CSL). The CSL code can be executed either in the official WSE simulator, provided through the Cerebras SDK, or directly on a CS appliance. Cerebras suggests to first develop the code on the simulator, and then run it on the appliance for making use of its performance.

Independently of the platform chosen (be it the simulator or the real WSE), in order to write code for the WSE there must be two parts that will be executed together: an **host code** and a **device code**. The host code is ran by the appliance as a support for coordinating the activity on the WSE, and it's written in Python. The device code is the code that will be executed directly on the WSE, and is instead written in CSL.

Conceptually, writing code for the WSE is not much different from writing code for a GPU: it's up to the programmer to perform the right `memcpy` calls (copying data from/to the device), to launch the kernels and to ensure synchronization between host and device. The difference lies mostly in the way PEs behave with respect to threads (PEs are independent, threads act in warps).

2.2.1 The Cerebras Software Language (CSL)

The CSL programming language is a proprietary, imperative and statically typed language from Cerebras, which provides a layer of abstraction over the WSE's Instruction Set Architecture (ISA)[2]. The compiler used to produce WSE-compatible code is based on the LLVM toolchain.

CSL is heavily inspired by the Zig language, borrowing part of its syntax for common elements such as control flow constructs and variable definition. There are however custom additions made from Cerebras, such the introduction of Data Structure Descriptors (DSD), programmable colors and custom built-in operations, recognized because of their leading `@`. All the code written in CSL is considered as device code.

In order to program in CSL, it's necessary to use the Cerebras SDK (which is currently at version 1.4.0). The SDK bundles together different tools, such as the compiler for CSL (`cslc`), a wrapper for running the Python host code (`cs_python`) and debug tools for inspecting the artifacts generated by the compiler. Moreover, the SDK provides a Singularity container for simulating a WSE, together with some libraries written by Cerebras to enhance the programming experience.

Code made for the WSE can be executed in one of three ways:

- at compile time (`comptime`): this includes global variables initialization, queues and task binding and `layout` construction;
- through function execution: functions behave exactly like most imperative programming languages, executing their content as soon as a function is called;

[2]The Cerebras ISA is currently not avaliable to the public through documentation, although it can be "inspected" by using the SDK visualization tool and other debug tools provided by Cerebras

- through task execution: tasks are a kind of non-interruptible function that get executed one at a time. In order to be scheduled, they are first placed in a queue, and when at least one of them is eligible for scheduling the CE will execute them, otherwise they remain in the queue.

2.2.2 Layout construction

In order to execute code on the WSE, the programmer must first declare the region of WSE that the program is going to be executed on. This is done through a `layout.csl` file, which contains compile-time instructions for building the environment.

In general, when defining a layout the programmer should define the following elements:

- any compile-time-known given parameter, passed by the compiler. These parameters can be of any supported type;
- the region of WSE over which the code will be ran. This is done through a mandatory builtin called `@set_rectangle()`;
- the code to be loaded on each PE in the region, done through a mandatory builtin called `@set_tile_code()`. The programmer must specify some code for **all** the PEs in the region; failing to do so will result in a compile error;
- the colors used for communication. They can either be specified in the layout at compile time or through specific commands at runtime; this last option however is usually reserved for changing the path of the color for some specific operations, making it good practice to specify the path beforehand at compile time.

All the code used for defining the layout is not stored on the WSE, and is used only for initialization. The actual code that will be executed is the one specified through `@set_tile_code()`.

2.2.3 Data, Local and Control Tasks

A task is a procedure that can be started only from the PE hardware, and that can't be interrupted until completion. Tasks can't be called, but activated. The difference with functions is that while a function executes immediately once called, a task can only be marked for execution when it gets activated, letting the WSE schedule it when possible. Tasks are the preferred way of executing code on the WSE, because they adapt more easily to the dataflow architecture.

There are three kinds of tasks:

- **local tasks**: also called self-activatable tasks, these tasks can only be activated by the PE through the builtin `@activate()`. Each local task is bound to a unique local task ID (in the range [8, 30]). A local task takes no input and can be scheduled only if it's not blocked either by the PE or by an asynchronous operation;

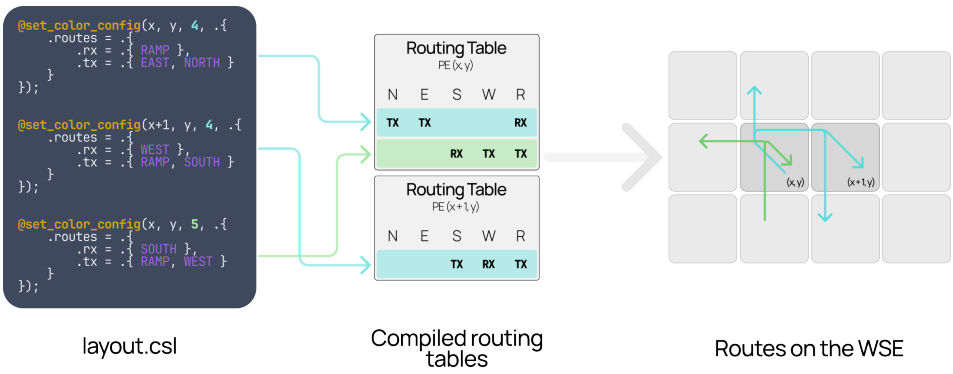

Figure 2.3. Example of color setup and how it is represented on the WSE

- **data tasks**: this kind of tasks is activated upon data reception, and are bound to a unique data task ID and to either an input queue on the WSE-3 or to a color on the WSE-2. An input queue is a temporary buffer stored in the router where data lies until the PE is ready for executing the relative data task. A data task can be activated if and only if the routable identifier to which it's bound is unblocked, meaning that it can receive data from other PEs. All data tasks have at least one input argument, which represents the incoming wavelet of data;

- **control tasks**: these tasks are activated upon reception of a specific kind of wavelet, which is called "control wavelet", and are bound to a specific control task ID (`CID`), in the range [0, 63]. A control task is activated when the following two conditions are both true at the same time:

  1) the PE receives a control wavelet containing the `CID` of the control task to be activated;
  2) the color over which the wavelet was sent is unblocked by the receiving PE.

Data tasks and control tasks are referred to as Wavelet-Triggered Tasks (WTT) because of their nature.

2.2.4 Colors

Data is able to flow within the WSE because of user-defined virtual communication channels called **colors**. Within the WSE, there are 24 available colors, and each of them is associated to an ID in the range [0, 23].

When defining the path of a certain color, for each PE we must specify the color ID that we are targeting, from which directions the color will receive data and to which directions it will send data. As mentioned previously, the possible directions towards which a color can both send and receive are `NORTH`, `SOUTH`, `EAST`, `WEST` and `RAMP`. Akin to a network router, the colors configurations for a PE are stored in a routing table, within a PE's dedicated routing memory.

While a color can't receive data from multiple sources, it can instead send to multiple PEs. This feature has been shown to be useful when designing utilities that must target all the PEs in a region. An example of this can be seen in Figure 2.3.

2.2.5 Data Structure Descriptors (DSD)

A useful feature of the WSE-3 that allows for vectorization capabilities are Data Structure Descriptors (DSDs), which are a compact representation of (possibly non-contiguous) chunks of memory. The usefulness of DSDs comes from the multiple operations that can be done on them, similarly to vectorized instructions on modern CPUs. DSDs are moreover used for sending and receiving data between PEs.

A DSD is composed of two elements: a type (which is one of `mem1d_dsd`, `mem4d_dsd`, `fabin_dsd` and `fabout_dsd`), and a set of properties. Among the types described before, the first two define a DSD in either 1 dimension or up to 4 dimensions, which is particularly useful when treating multi-dimensional data such as images. The `fabin_dsd` and `fabout_dsd` types are used for defining input and output chunks of data, and are both associated to either a queue (if on a WSE-3) or to a color (if on a WSE-2).

DSDs can be defined either by specifying each property singularly, or by defining a tensor access expression, which bundles automatically all the DSD's properties. A tensor access expression comes in the following form:

```
|induction variables|{length} -> <base address>[expression]
```

where:

- `induction variables` is a set of variables to be used for defining how to access to data, up to 4 when using a 4D DSD, and only 1 when using a 1D DSD;
- `length` is a set of comp-time strictly positive integers defining the number of times to iterate for each defined induction variable;
- `base address` is the starting address of an array upon which the DSD is going to be defined;
- `expression` is the mathematical pointer expression used for defining the access to the array.

DSD properties can be changed at runtime either by editing the singular properties or by defining again the DSD. The importance of DSD lies in the set of available operations that can be done with them, and the possibility of executing them, in some cases, asynchronously. Examples of operations that can be done are `@fadds()`, which performs the addition between two DSDs of 32-bits floating point numbers (`f32`), `@fmaxs()`, which computes the element-wise max operation between two `f32` DSDs, and so on...

In order to store information about DSDs, the WSE adopts a small 256 B cache [23]. The values of a DSD are instead stored into physical registers called Data Structure Registers (DSR). There are three register files, `dest`, `src0` and `src1`, which are all

employed when performing operations on DSDs. In most of the cases, DSRs can be considered as opaque, and all the DSD operations will perform DSRs operations automatically; however, the SDK exposes a set of APIs for better controlling the DSRs.

2.2.6 I/O communication

Communication between the host and the WSE is performed through the `SdkRuntime`, which is a host-sided runtime, and the `memcpy` library for the device. These two elements are both bundled within the Cerebras SDK.

`SdkRuntime` manages all the data transfers between the host and the device, but on the WSE itself the library responsible for managing data and routing it to the right kernel is `memcpy`. On the WSE, data can flow in and out only through some I/O channels, which are placed on the east and west sides of the chip and are spaced roughly 16 rows apart. The WSE-3 can count 62 total I/O channels. In order to work properly, the `memcpy` library requires specific portions of the WSE, being a 1 PE halo around the whole WSE, 3 PE columns on the west side and 2 PE columns on the east side. These columns act as buffers for the data while the WSE is busy, and we can choose to increase the dimension of the buffer columns at compile time thanks to specific flags. The `SdkRuntime` is however able to use only 16 I/O channels simultaneously.

The `SdkRuntime` host runtime supports the upload and download of data into two distinct modes: copy and streaming. In this work, we'll only use the copy mode, but final considerations will take into account the streaming mode too. In order to use copy mode, the user must do the following:

- from the device code, it must define all the variables that are exposed for importing/exporting, and whether the variable is mutable or not by the host. All the exported variables (both from the host and the device) must be arrays: for instance, if we wanted to export a constant, on the WSE the destination must be a one-element array;
- from the host code, it must define all the symbols to which data will be copied to/from. Then, after launching the simulator, the user can perform all the `memcpy` calls needed. When launching a kernel on the WSE, the next memcpy will be performed after all the PEs return command to the host.

2.2.7 Program flow

During execution, a sample Cerebras program usually goes through a static set of stages: these can slightly change depending on which features the program uses (for instance, if a streaming `memcpy` is employed), but they are similar both on the simulator and on the WSE. An overview of which stages the program goes through is here provided, and a timeline is available in Figure 2.4. The stages are the following:

1) after compiling the device program, the SDK runs the host code, initiating the `SdkRuntime`, retrieving the exported symbols from the compiled code and initiating any user variable;

2) the host starts copying data over to the device by calling the `memcpy_h2d()` function, which can either be synchronous or asynchronous;

3) after copying all the needed data, the host launches the kernel through the `launch()` method of the `SdkRuntime` instance. The launch is asynchronous, making the host execute the rest of its program. The host must ensure not to end the instance of `SdkRuntime` before the device finishes executing the kernel. This is usually ensured through a synchronous device-to-host `memcpy`, which will execute only after the device returns full control to the host;

4) the kernel starts executing on the WSE, and after finishing all the PEs will have to call the `unblock_cmd_stream()` function, which will return full control to the host;

5) the host issues one or more device-to-host `memcpy` operations, retrieving all the data from the WSE. After that, the `SdkRuntime` will be stopped, allowing the Python code to gracefully finalize and return any output to the user.

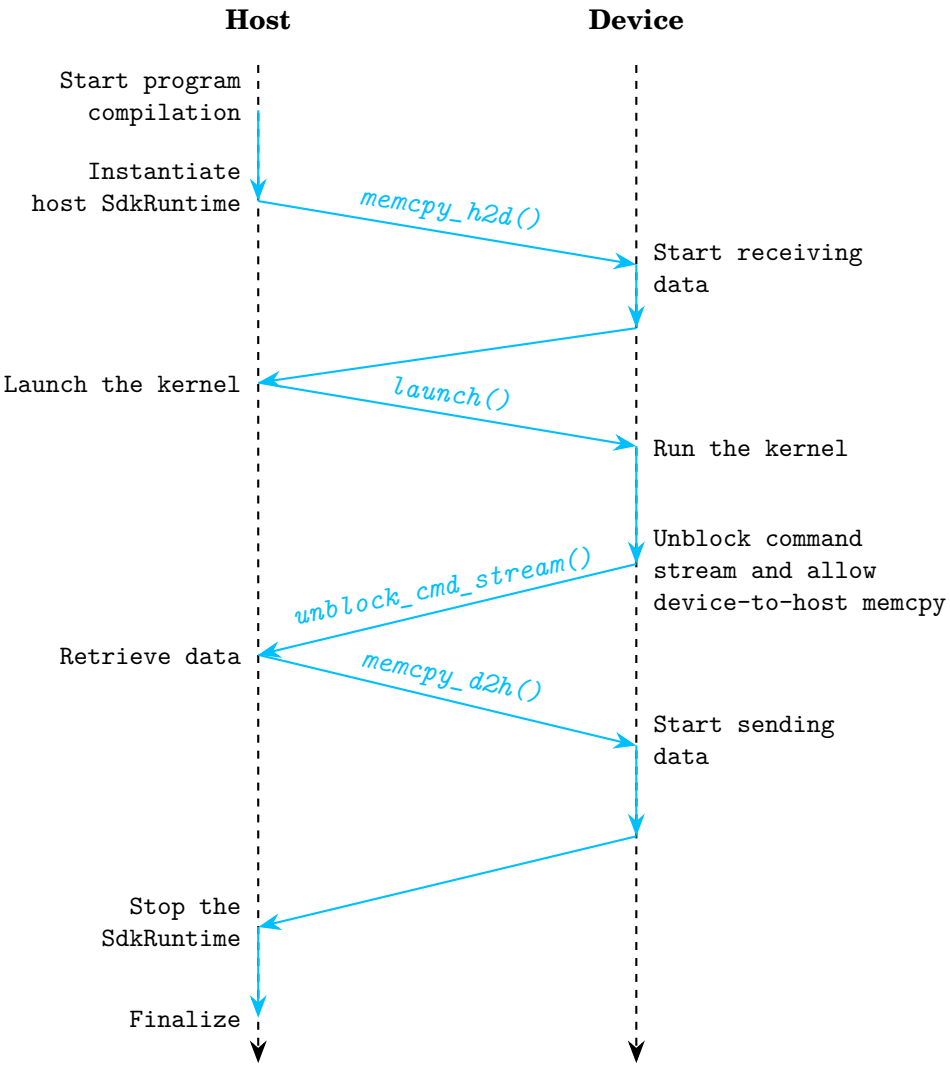

Figure 2.4. Visualization of a simple Cerebras program flow and the interactions between host and device. All `memcpy` calls are considered as synchronous, meaning that in order to return control to the host they must first end. Once a `memcpy` is done, an arrow is shown to indicate that the call has finished executing, and that the host has full control again

Chapter 3
The CereMath library

As discussed in the previous chapter, there are two primary methods for writing code for the WSE: the user can either use official PyTorch (or TensorFlow) bindings, which run directly on the appliance, or write code natively in CSL. The first approach is preferred for AI-related tasks (such as model training or performing inference), primarily because it enables the execution of pre-existing code on the WSE with minimal effort. However, this method proves to be less suitable for performing low-level optimization.

This limitation arises from the lack of public availability of the CSL/Assembly code for the various PyTorch (or TensorFlow) bindings, making the identification of precise optimization areas challenging, and requiring users that don't want to write in CSL to trust the bindings provided by Cerebras. Specifically, crucial details remain unknown, such as how the operations function internally, the specific regions of the Processing Element (PE) they occupy, and their interconnection mechanisms.

Conversely, this limitation is less present in more traditional architectures (such as CPUs and GPUs), for which there is a wider availability of open-source libraries that have become the *de facto* standard for executing common mathematical operations. Some examples include the LAPACK and CuBLAS libraries, both of which adhere to the Basic Linear Algebra Subroutines (BLAS) specification.

To date, there have been no documented attempts to develop dedicated BLAS-like libraries specifically for the WSE. This is likely due to the following reasons:

- first, as the WSE is a "new" architecture utilizing a proprietary ISA, it's not possible to go too much in depth with respect to possible optimizations. Although Cerebras has made efforts to document its hardware and SDK, numerous details remain undisclosed, with some information being occasionally clarified on the developers' forum;
- second, unlike traditional Von Neumann architectures, where data is typically structured to exploit hardware features (such as vectorization), dataflow architectures require the program to be tailored around the flow of data. This is because reshaping data at runtime would stall dataflow computations, hence reducing the overall throughput of the program. Consequently, defining fixed kernels is challenging, as specific adaptations are required depending on how data is routed through the WSE.

Motivated by these challenges, this work presents the implementation of four standard Machine Learning kernels that, under specifc layout assumptions, can be considered as general-purpose kernels. The proposed kernels are **Convolution**, **Pooling**, **GEMV** and the **GEMM** operations.

Type	Symbol	Description
Variables	n, α, W	Non-bold characters, small or capital, are constants
	$\mathbf{x}$	Small, bold characters are column-vectors
	$\mathbf{A}$	Capital, bold characters are matrices
	$\mathbf{a}_i^{(j)}$	Vectors/arrays, differentiated by the superscript (j), in position i. If the array is denoted as $\mathbf{t}$, then it's a temporary array
	n	Amount of rows of PEs
	r	Rows of a vector/matrix to be distributed to each PE
	s	Stride of Convolution and Pooling kernels
Operations	$\mathbf{A} \times \mathbf{B}$	Cross product between $\mathbf{A}$ and $\mathbf{B}$
	$\mathbf{A} \odot \mathbf{B}$	Hadamard product between $\mathbf{A}$ and $\mathbf{B}$
	$\mathbf{a}_i \longleftarrow \alpha$	Store of α in vector/array $\mathbf{a}$ in position i

Table 3.1. Legend of the notation used

In the following sections, mathematical formulations will be frequently employed to formally describe the underlying operations. To ensure clarity and consistency, a summary of the mathematical notation used is provided in Table 3.1.

3.1 The State of the Art of ML kernels

Matrix-matrix and matrix-vector operations have been extensively optimized over time, particularly in the early stages of computer science. As foundational operations across a wide range of applications (such as simulations, data analysis, etc...), developing fast, high-performing implementations has always been crucial. Consequently, while numerous implementations of these operations exist, the majority adhere to a common standard known as BLAS [21, 6].

The BLAS standard defines a set of mathematical routines that serves as fundamental building blocks for programs, and is organized into three levels:

- **Level 1** describes scalar, vector and vector-vector operations, and was first released in 1979 [21];
- **Level 2** introduces matrix-vector operations. It was released as an update nearly a decade after the original standard, incorporating several optimizations specifically designed for vector processors [12];
- **Level 3** addresses matrix-matrix operations. Released two years after Level 2, it aimed to provide support for more complex operations, which were previously executed by chaining multiple matrix-vector computations and did not scale properly [11].

The specific BLAS operations analyzed and ported to the WSE in this work are the GEMV and GEMM operations, which belong to Levels 2 and 3 respectively. The GEMV represents a general matrix-vector multiply-and-accumulate operation: given $\alpha, \beta \in \mathbb{R}$, $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{b}, \mathbf{y} \in \mathbb{R}^n$, GEMV is mathematically described as [12]:

$$\mathbf{y} = \alpha \mathbf{A} \times \mathbf{x} + \beta \mathbf{b} \tag{3.1}$$

Conversely, the GEMM operation represents a general matrix-matrix multiply-and-accumulate. Given $\alpha, \beta \in \mathbb{R}$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times o}$ and $\mathbf{C} \in \mathbb{R}^{m \times o}$, GEMM is defined as [11]:

$$\mathbf{C} = \alpha \mathbf{A} \times \mathbf{B} + \beta \mathbf{C} \tag{3.2}$$

Although described by a single standard, the implementation of these operations greatly differs depending on the underlying architecture. For instance, Intel provides the Intel oneAPI Math Kernel Library (oneMKL) for its CPUs, just like AMD provides AOCL-BLIS; NVIDIA and AMD supply proprietary versions for their respective GPUs, such as NVIDIA's cuBLAS. In contrast to these architecture-specific implementations, GNU provides a platform-agnostic version of the BLAS standard, called CBLAS.

The proliferation of these different versions stems from the need to exploit the specific hardware resources unique to each vendor's product. To assess the Cerebras-ported kernels, performance comparisons will be conducted with NVIDIA's cuBLAS library, as the evaluation environment consists of NVIDIA GPUs.

Although for the GEMV and GEMM operations a state-of-the-art reference can be found for traditional architectures, this can't be said regarding the Convolution and Pooling kernels. This is primarily because these operations specifically belong to the field of Deep Learning, where the optimization focus is memory management. Although computational throughput remains important, training AI models is inherently a memory-bound task, making data reuse and efficiently managing limited GPU memory more crucial. The optimization of these kernels therefore relies on two main entities: Deep Learning framework developers and hardware accelerator manufacturers, who also maintain the mathematical libraries for their respective devices.

3.1.1 Available implementations for the WSE

Since the WSE is primarily marketed as an AI-first architecture, all of the previously mentioned operations have already been implemented on the WSE to some extent, particularly GEMM and GEMV. However, these implementations are mostly related to specific AI models and vary depending on the use case. Consequently, the WSE currently lacks a standardized library.

An example of such adaptation is presented in the WaferLLM paper, which introduces the MeshGEMM and MeshGEMV kernels [16]. Those kernels employ a tiling approach, utilizing a region of $n_{\text{sub}} \times m_{\text{sub}}$ PEs to process the tiles. The kernels demonstrate exceptional performance with large tile sizes, outperforming the standard implementation provided by Cerebras, which is a porting of the SUMMA algorithm [28]. Specifically, the MeshGEMM kernel requires roughly half the execution time of

SUMMA for tiles of 2,000 elements; however, performance gradually degrades as the size of the tiles increases, likely due to communication overhead. For the MeshGEMV kernel, this performance advantage is maintained even at larger sizes (e.g., 8,000 elements). Nevertheless, according to the authors, further increases in element size eventually lead to performance degradation, again attributable to communication overhead.

While these kernels are effective and deliver strong performance, a significant downside is that their design inherently requires a large number of PEs, making them less suitable in a general-purpose context. Although the large size helps overcome the limited memory capacity of individual PEs, it also requires efficient communication collectives. In a broader context, kernels that occupy a substantial portion of the WSE inevitably restrict the number of concurrent operations that can be executed. Therefore, this work focuses on developing small, compact kernels, at the cost of having to use less data multiple times.

3.2 Implementation of CereMath's kernels

The design of the kernels proposed in this work is driven by the following core objectives:

- **compactness**: minimizing the size of individual kernels allows the placement of multiple identical kernels across the WSE, enabling workload distribution and parallelization.
- **modularity**: since the WSE achieves performance through data flow between PEs, it's important to ensure kernel modularity, to facilitate the connection between them.

To achieve these goals, this work proposes a kernel structure common across all implemented functions, composed of four main sub-operations:

- `op0`: the initial sub-operation acts as data buffer, or as a receiver when multiple kernels are interconnected. To facilitate this, the program's memory space must be minimized to maximize the amount of data stored within the PE's SRAM. This stage can also perform small data manipulations before passing the data to the next sub-operation. Data is transmitted as vectors, typically as individual rows of the initial dataset;
- `op1`: the second sub-operation executes a sequence of operations on the row received from `op0`. It requests new data from the previous PE upon the completion of each execution cycle. Once a row is processed, the obtained data is passed to the next sub-operation;
- `op2`: the third sub-operation may perform preliminary small computations on the input data before executing either a reduce or a gather operation to accumulate all the partial results. Due to communication overhead, this step is usually the one that can take the longest amount of time. After collecting all the data, the results are passed to the final sub-operation;

- `op3`: the last sub-operation is responsible for storing the received data, either retaining it for host retrieval or forwarding it to another kernel. This sub-operation has, moreover, the task of shutting down the WSE upon completion of all the kernels; this task will be described in the final section of this chapter.

The proposed structure achieves compactness by fitting, into a rectangular grid of PEs with a fixed width of 4 PEs and variable height defined by the user, any of the four proposed kernels. In this manner, increasing the number of rows directly increases the number of PEs participating in the computation of a single result.

All the kernels here presented are designed to work with IEEE 754 32-bits floating point numbers [4]. This design choice was made because the functions from the Cerebras' collectives library `Collectives2D` were not available for lower precision formats. As a result, the space used by the data is greatly increased, limiting the amount of data that can be stored within the individual sub-operations.

For communicating, the sub-operations use a restricted amount of colors: 7 for the Convolution and the Pooling kernels, and 8 for the GEMV and GEMM kernels. Beyond the kernel boundaries, these colors can be freely reused except for one, which is the one responsible for tramsmitting the exit signal across the entire WSE.

Across all kernels, the sub-operations behave in a pipeline-like structure to minimize inactivity moments and maximize overall throughput. Furthermore, several core mechanics remain consistent throughout the kernels, specifically:

- sub-operations `op0`, `op1` and `op2` wait for an exit signal from `op3` upon completing their execution to terminate and return back control to the host;
- `op1` requests new data from `op0` via a command wavelet signal, transmitted in the background prior to processing the just-received data. Concurrently, `op0` continuously listens for any incoming request;
- sub-operations `op0` and `op3` mainly serve as input and output buffers for the data. Upon receiving the complete dataset, `op3` broadcasts an exit signal to all the PEs, to ensure graceful termination. Subsequently, the host retrieves the results via a `memcpy_d2h()` call;
- for explanatory purposes, a simplified environment will be considered in which the kernels operate in isolation, neither receiving nor transmitting data to any other kernel.

3.2.1 Convolution

In the Deep Learning scenario, Convolutions have historically been among the most used computational kernels, serving as the basis of Convolutional Neural Networks (CNN). After Yann LeCun proposed a first version of CNN, called LeNet-5 [22], numerous other CNN variants emerged. Notable examples include AlexNet [20], which started the Deep Learning era; GoogLeNet, with introduced the inception module [27]; ResNet, which adapted residual connections for better gradient propagation [17]. More recently, following the introduction of the Transformer architecture by Vaswani [29], CNNs were surpassed by Visual Transformers (ViT). ViTs require

fewer computational resources to pre-train (but not to both train and make inference) and have demonstrated better performances overall compared to CNN models like ResNet [13] over large datasets, beating what was previously considered as the State-of-the-Art (SotA) in image classification.

Although ViTs achieve better performances on large datasets, this advantage comes at the cost of higher number of trainable parameters and significantly greater memory requirements compared to CNNs, which remain a suitable option for smaller datasets and devices with restricted hardware capabilities [32]. Given the limited amount of memory capacity per PE (and on the WSE in general, which can hold at most $\simeq 44$ GB of data by summing all the 48 kB of its 900,000 PEs) and the current scarcity of Computer Vision research conducted on the WSE (especially with CNNs), this work proposes a foundational Convolution kernel tailored specifically for the WSE.

Considering C_{in} and C_{out} as respectively the amount of input and output channels, given a part of a data matrix $\mathbf{I}_{\text{part}} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$[1], a filter matrix $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times f \times f}$ and a bias vector $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$, a Convolution of a given output channel C_{out} is mathematically described as the sum between a value from the bias vector and the summatory of the values obtained from the Hadamard product[2] between a filter matrix's channel and an image's channel, for all the input channels:

$$\text{Conv}(\mathbf{I}_{\text{part}}, C_{\text{out}}) = \mathbf{b}(C_{\text{out}}) + \sum_{i=1}^{C_{\text{in}}} \mathbf{W}(C_{\text{out}}, i) \odot \mathbf{I}_{\text{part}}(i) \tag{3.3}$$

By following the initially set design guidelines, especially regarding compactness, the computations are split among r rows, where r is ideally equal to the size of the filter (so $r = \mathbf{W}_W = \mathbf{W}_H$): this means that the kernel will have a total size of $4 \times r$. Starting from Equation 3.3, the computation that each row in the kernel will be responsible to carry on is defined as:

$$\text{Conv}(\mathbf{I}_{\text{part}}, C_{\text{out}}, \text{PE}_y) = \underbrace{\mathbf{b}(C_{\text{out}}) + \underbrace{\mathbf{W}(C_{\text{out}}, \text{PE}_y) \odot \mathbf{I}_{\text{part}}(\text{PE}_y)}_{\text{Step 1}}}_{\text{Step 2}} \tag{3.4}$$

In order to achieve this, a single Convolution kernel takes as input the following data: r rows of the input image $\mathbf{I} \in \mathbb{R}^{C_{\text{in}} \times r \times W}$, the filter matrix $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times r \times r}$, the bias vector $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$, the sizes C_{in}, C_{out}, r, W and the stride s of the convolution.

The 4 sub-operations are then organized as follows:

- in `op0`, the host stores a row of the image matrix $\mathbf{I}$, which will send the data over to `op1`. For moving the data, we take advantage of the WSE's DSDs, and send just a portion of the image $\mathbf{I}_{\text{part}}$ at a time: in order to achieve this, a tensor access expression has been used:

[1] For better explaining the formula, the $\mathbf{I}_{\text{part}}$ symbol will be used to denote a portion of the image over which a convolution is being executed at any given timestep. For indicating the whole image matrix, the symbol $\mathbf{I}$ will be used instead

[2] The Hadamard product is the element-wise product between two matrices, and as per Table 3.1 is indicated with the symbol $\odot$. The two matrices multiplied must have the same size

```
tens_expr = |ch, conv_idx|{ch_in, conv_size} -> x[ch * I_width + conv_idx]
```

For this expression, the DSD will take from all the channels (as dictated by `ch_in`) the first `conv_size` consecutive elements. In order to shift within the array, the program increases the offset of the DSD access pointer at each image patch sent. By increasing it by s elements, then it's possible to access to the whole image, so long that the following condition is respected[3]:

$$\frac{W-r}{s}+1 = W_{\text{out}} \mid W_{\text{out}} \in \mathbb{N} \tag{3.5}$$

- in `op1`, the host stores the whole filter matrix $\mathbf{W}$, and upon receiving of a full patch of image from the previous operation, it activates the data processing task. Said task produces a vector of size $\mathbb{R}^{C_{\text{out}}}$, which contains in each entry the operation "Step 1" of Equation 3.4:

$$\mathbf{t}_i^{(1)} \longleftarrow \mathbf{W}_i \odot \mathbf{I}_{\text{part}}, \quad \forall\, i \in C_{\text{out}} \tag{3.6}$$

  The operations are performed with a mix of `@fadds()` and `@fmuls()` builtins. When this vector is ready, it is passed on to the next operation, and the sub-operation will either wait for a new piece of data from `op0` or for the exit signal from `op3`;

- in `op2`, the host stores the whole bias vector $\mathbf{b}$ on the last PE of the `op2` column. After all the PEs receive a vector from the previous operation, a `reduce_fadds()` collective with accumulate operation starts, which will send to a single PE (in our case, the last) all the separate vectors. After the `reduce` is complete, the receiving PE sums the bias vectors to the reduced vector (as per "Step 2" in Equation 3.4), which is finally sent to `op3`:

$$\mathbf{t}^{(2)} \longleftarrow \mathbf{b} + \underbrace{\sum_{i=0}^{n} \mathbf{t}_i^{(1)}}_{\texttt{reduce\_fadds()}} \tag{3.7}$$

- `op3` receives all the partial results and sends, after receiving all the data, a control packet to all the PEs, signaling that the kernel has finished executing.

3.2.2 Pooling (Max and Average)

Similarly to the Convolution kernel, the Pooling operation started being extensively used primarily in Deep Learning scenarios. For the same reason for which a Convolution kernel has been proposed, here is presented an implementation of the Pooling kernel. This work proposes two versions of the Pooling kernel, which are the max Pooling (MaxPooling) and the average Pooling (AvgPooling). The algorithm for the Pooling kernel is very similar to the one of the Convolution, with the difference that it holds overall less data (since both filter matrix and bias vectors are not needed).

[3] In Equation 3.5, W can be replaced with H to test the condition vertically on the image

Through the development of this kernel, some issues were encountered with the `Collectives2D` library from Cerebras. Especially, in order to make the MaxPooling kernel, there was the need of introducing a `reduce` collective with the max() operation. In a further section, it will be explained which modifications were made, the way the library got modified, and the reason why they were needed.

Given a part of a data matrix $\mathbf{I}_{\text{part}} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$, a MaxPooling kernel is mathematically described as the max value in a given channel C_i of $\mathbf{I}_{\text{part}}$:

$$\text{MaxPool}(\mathbf{I}_{\text{part}}, C_{\text{i}}) = \underbrace{\max_{j \in H} \underbrace{\max_{k \in W} \mathbf{I}_{\text{part}}(C_{\text{i}})_{j,k}}_{\text{Step 1}}}_{\text{Step 2}} \tag{3.8}$$

Instead, the AvgPooling kernel is mathematically described as the average value of a given channel C_i of $\mathbf{I}_{\text{part}}$:

$$\text{AvgPool}(\mathbf{I}_{\text{part}}, C_{\text{i}}) = \underbrace{\frac{1}{H \cdot W} \cdot \sum_{j=0}^{H} \underbrace{\sum_{k=0}^{W} \mathbf{I}_{\text{part}}(C_{\text{i}})_{j,k}}_{\text{Step 1}}}_{\text{Step 2}} \tag{3.9}$$

As per the Convolution kernel, with the Pooling kernel too it's ideal to split the rows of the input image among r rows of the PE, having thus a kernel able to process $r \times r \times C_i$ elements at each stage. The two Pooling kernels are very similar between each other, differing only in the computations done and in the lower amount of inputs needed. Indeed, both Pooling kernels need just the input image $\mathbf{I} \in \mathbb{R}^{C_{\text{in}} \times r \times W}$, the sizes C_{in}, r, W and the stride s. Given the similarity of the two kernels, their sub-operations will be explained together:

- `op0` performs the duty of initial buffer and receives in input only the input image $\mathbf{I}$. In order to send the data, it uses the same tensor access expression mentioned in the Convolution kernel, which sends $r \times r \times C_i$ elements per request;
- `op1` presents various changes in the inputs received and the operations performed: since no filter is needed in the Pooling operations, this sub-operation takes no input from the host. The computations too are different, especially depending on the type of Pooling that is used:
  - in the case of a MaxPooling, it computes the maximum element between all the values in a channel, for all the channels ("Step 1" of Equation 3.8, repeated for all the channels):

$$\mathbf{t}_i^{(1)} \longleftarrow \max_{k \in r} \mathbf{I}_{\text{part}k}, \quad \forall\, i \in C_{\text{in}} \tag{3.10}$$

  - in the case of an AvgPooling, it accumulates through a sum all the values of a channel ("Step 1" of Equation 3.9, repeated for all the channels):

$$\mathbf{t}_i^{(1)} \longleftarrow \sum_{k=0}^{r} \mathbf{I}_{\text{part}k}, \quad \forall\, i \in C_{\text{in}} \tag{3.11}$$

After performing the aforementioned computations, the sub-operation produces an array of size $\mathbb{R}^{C_{\text{in}}}$, which is then passed to `op2`;

- in `op2`, similarly to `op1`, there are no inputs coming from the host, since the bias vector is not needed. Akin to the Convolution kernel, the partial results of all the rows passed from `op1` need to be collected by one single PE, and this is done in one of two ways, depending on the type of Pooling:
  - in the case of a MaxPooling, a `reduce_fmaxs()` collective will be called, computing the maximum value for each position of the reduced array. This collective was not part of the original `Collectives2D` library, and has been created in order to allow this kernel to work. The collective performs the "Step 2" of Equation 3.8:

$$\mathbf{t}_i^{(2)} \longleftarrow \texttt{reduce\_fmaxs}\left(\mathbf{t}_i^{(1)}\right), \quad \forall\, i \in C_{\text{in}} \tag{3.12}$$

  - in the case of an AvgPooling, a `reduce_fadds()` collective will be used, reducing the partial arrays with a sum operation. The reduce computes "Step 2" of Equation 3.9. After the reduce operation completes, the final array is then divided by the amount of elements considered at each Pooling iteration, which is r^2:

$$\mathbf{t}_i^{(2)} \longleftarrow \frac{1}{r^2} \cdot \texttt{reduce\_fadds}\left(\mathbf{t}_i^{(1)}\right), \quad \forall\, i \in C_{\text{in}} \tag{3.13}$$

  After completing the collective, the receiving PE will pass the data to `op3`, and will wait for either new data or an exit signal;
- `op3`, just like in the Convolution kernel, acts as a buffer for storing the final result. After finishing, it will send via a control packet an exit signal to all the PEs, and the data will be retrieved via a `memcpy_d2h()` call.

3.2.3 GEMV

As described previously, the GEMV kernel is a Level 2 kernel from the BLAS standard, which has particular relevance in Machine Learning settings as well: an example is the evaluation of one layer of a multi-layer perceptron, which is computed as $\mathbf{y}_i = \mathbf{W}\mathbf{x} + \mathbf{b}$.

In the GEMV kernel, the original equation (3.1) is split between all the sub-operations, in order to get the most out of the dataflow capabilities of the WSE. For this kernel, the inputs needed are α, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, β, $\mathbf{b} \in \mathbb{R}^m$, the sizes m, n and r. The kernel will then take as input, for each row, $\mathbf{A}_{\text{part}} \in \mathbb{R}^{r \times n}$

The four sub-operations perform thus the following tasks:

- `op0` receives α and $\mathbf{A}_{\text{part}}$ from the host, and before sending them to `op1` it computes $\alpha \cdot \mathbf{A}_{\text{part}}$ with the `@fmuls()` instruction;

- `op1` receives $\mathbf{x}$ from the host, and $\alpha \cdot \mathbf{A}_{\text{part}}$ from `op0`. Upon receiving one row of data i, it computes

$$\mathbf{t}_i^{(1)} \longleftarrow \sum_{k=0}^{n} \alpha \cdot \mathbf{A}_{\text{part}\,i} \times \mathbf{x}_i, \quad \forall i \in r \tag{3.14}$$

  Once $\mathbf{t}_i^{(1)} \in \mathbb{R}^r$ fills, the data stored within it will be sent over to `op2`. After sending the data, the array is overwritten by the next row of data $i+1$ from `op0`;

- `op2` executes, as soon as the program starts, the operation $\beta \cdot \mathbf{b}$, and then waits until it receives $\mathbf{t}_i^{(1)}$. Upon receival, the sub-operation performs the following process:

$$\mathbf{t}_i^{(2)} \longleftarrow \mathbf{t}_i^{(1)} + \beta \cdot \mathbf{b} \tag{3.15}$$

  After that, the sub-operation gathers all the temporary arrays using a `gather()` collective, and upon completion it sends the gathered data to `op3`;

- `op3` acts as a output buffer, hence counting all the received data. Upon receival of all data, it terminates the program and returns control to the host.

3.2.4 GEMM

The GEMM kernel, a Level 3 operation from the BLAS standard, is mathematically represented as in Equation 3.2, and is also extensively employed in Machine Learning and Deep Learning settings, usually for computing the results of the feed-forward multi-layer perceptron placed at the end of many used architectures.

As per the GEMV kernel, here too each operation of Equation 3.2 has been assigned to a different sub-operation. The inputs for the kernel are α, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times o}$, β, $\mathbf{C} \in \mathbb{R}^{m \times o}$, the sizes m, n, o and r.

The kernel works as follows:

- `op0` receives α and $\mathbf{A}_{\text{part}}$ from the host, and computes $\alpha \cdot \mathbf{A}_{\text{part}}$ with a single `@fmuls()` instruction. Upon completion, the data is sent to `op1`;

- in `op1`, each PE receives the $\mathbf{B}$ matrix from the host and computes $\alpha\mathbf{A} \times \mathbf{B}$ in the following way: for each incoming element (recall that the WSE sends data in single 32 bits wavelets), it is multiplied with a row of B, and the result is then stored in a temporary array. This is because in matrix multiplication, each row of $\mathbf{A}$ must be multiplied eventually with each column of $\mathbf{B}$, producing a single accumulated scalar; so given an arbitrary row of $\mathbf{A}$, multiplying it for all $\mathbf{B}$ would be akin to perform a simple matrix-vector multiplication. The result would be a row of $\mathbf{AB}$, stored in the same position as the row of $\mathbf{A}$ that got multiplied to $\mathbf{B}$. This can be observed in Equation 3.16. The result is then sent to the next sub-operation;

$$\underbrace{\begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \mathbf{a_{i,1}} & \cdots & \mathbf{a_{i,n}} \\ \vdots & \ddots & \vdots \\ a_{m,1} & \cdots & a_{m,n} \end{bmatrix}}_{\mathbf{A} \in \mathbb{R}^{m \times n}} \times \underbrace{\begin{bmatrix} b_{1,1} & \cdots & b_{1,o} \\ \vdots & \ddots & \vdots \\ b_{n,1} & \cdots & b_{n,o} \end{bmatrix}}_{\mathbf{B} \in \mathbb{R}^{n \times o}} = \underbrace{\begin{bmatrix} ab_{1,1} & \cdots & ab_{1,o} \\ \mathbf{ab_{i,1}} & \cdots & \mathbf{ab_{i,o}} \\ \vdots & \ddots & \vdots \\ ab_{m,1} & \cdots & ab_{m,o} \end{bmatrix}}_{\mathbf{AB} \in \mathbb{R}^{m \times o}} \tag{3.16}$$

- op2 receives β and $\mathbf{C}_{\text{part}}$ from the host (where $\mathbf{C}_{\text{part}} \in \mathbb{R}^{r \times o}$ represents a sub-matrix of r rows of $\mathbf{C}$), and before accepting data it performs $\beta \cdot \mathbf{C}_{\text{part}}$. Upon completion, it unblocks the communication channels with op1 and waits. At each received wavelet, the sub-operation computes the sum between the element of $\mathbf{AB}$ and its corresponding element of $\mathbf{C}_{\text{part}}$. After all the elements have been received, a `gather()` collective is performed, sending to a single PE all the data;

- op3 acts as a final buffer, holding the gathered data from op2 until the host collect them, and performing the WSE teardown.

3.2.5 Adaptation of the `Collectives2D` library

In all the kernels presented, op2 always had the task of collecting the data produced by all the rows: internally, the kernels make use of the `Collectives2D` library provided by the Cerebras SDK, which provides functions that are heavily inspired by the Message Passing Interface library (MPI). The `Collectives2D` library comes with 4 collectives, being `broadcast()`, `scatter()`, `gather()` and `reduce_fadds()`. This library was designed such that only one collective per axis can be performed at a time (so either on the x or y axis). The axis is defined when the library initializes.

In the Pooling kernel, it was mentioned a `reduce_fmaxs()` collective, which is not part of the original library. This function was added to the library in order to make the Pooling kernel work. The code for this collective is the same as the one of `reduce_fadds()`, but with the difference that it uses the `@fmaxs()` builtin to compare the incoming data instead of the `@fadds()`. This could potentially be done with any of the 32 bits builtin operations for DSDs, opening the door to a wider range of computations that could be done. For the sake of this work, only this collective was added.

The `reduce_fmaxs()` collective is not the only feature that was added. While producing the kernels, an odd behaviour was observed: if multiple kernels were placed on the same column, then they would always stall at op2. Displacing them by one PE was enough to make them execute, so long that on a given column no other collective was being used. While this is a quick fix and may work if the user wants to use a small amount of kernels, all displaced on the WSE, it's not admissible in cases where it's needed to put a large amount of kernels all nearby.

Upon inspection of the inner workings of the library, it has been discovered that during the initialization phase of the library, the coordinate system used is global, and the collective assumes that the user is going to use an entire column (or row. For clarity, assume that a column is being considered) for just one collective at a time. This clashes with the possibility of stacking different kernels together. In order to fix this, the coordinate system must be changed, making it local to each kernel. For doing this, the following modifications were made[4]:

[4]This issue was raised on the Cerebras Developers Forum and, at the time the author is writing this thesis, it's still under discussion. The whole discussion can be found at https://discourse.cerebras.net/t/collectives-library-stall-about-using-collectives-simultaneously/809

- in the `Collectives2D/params.csl` file, the `get_params()` function, which creates a struct containing all the needed data for using the library, had to be modified so that to not only take the coordinates of the PE that will use the library, but also the region of WSE on which the collectives will take place:

```
1 fn get_params(
2     Px: u16, Py: u16,                       // Coordinates of calling PE
3     coll_x_start: u16, coll_y_start: u16,   // Start of collective (x, y)
4     coll_width: u16, coll_height: u16,      // End of collective (x, y)
5     ids: comptime_struct) comptime_struct { ... }
```

  In the code shown, lines 3 and 4 represent the additions to the function parameters. Such values are used to build the struct that the library will use when calling the collectives. The returned struct is a concatenation of two structs (one per dimension), which have been both modified as follows:

```
const x_dim_info = .{
    .pe_min = (Px == coll_x_start),
    .pe_max = (Px == coll_x_start + (coll_width - 1)),
    .NUM_PES = coll_width,
    .DIM_ID = 0, // x-dimension has an ID of 0
    .x_start = coll_x_start
};

const y_dim_info = .{
    .pe_min = (Py == coll_y_start),
    .pe_max = (Py == coll_y_start + (coll_height - 1)),
    .NUM_PES = coll_height,
    .DIM_ID = 1, // y-dimension has an ID of 1
    .y_start = coll_y_start
};
```

  Originally, the `.NUM_PES` field had as value one of the dimensions of the WSE region used when calling `@set_rectangle()` in the layout file (for the `x` dimension it used the rectangle's width, for the `y` dimension the height). Now, it just takes the `coll_width` or `coll_height` parameter, so that to restrict the library to a user-defined region. In both structs, the starting coordinates of the library have been added. These are used later in the library for placing the colors and performing the teardown;

- in the `Collectives2D/pe.csl` file, there is a function that must be called once for initializing the library, preferrably at the beginning of the code (akin to `MPI_Init()`, the initialization function of the MPI library). Such function unpacks the struct created by `params.csl`, and prepares all the variables that are not compile-time known. Here, all the local coordinates are also computed, which will then be employed throughout the file.

  Specifically, two new variables have been added: `local_id`, which is either the `x` or `y` coordinate of the calling PE with respect to the collective region (for instance, suppose that the collective takes part in the region with top-left corner at $(4,5)$ and bottom-left corner at $(4,9)$: when PE $(4,8)$ initializes the library on the `y` axis, it will have that its `local_id` is equal to 3), and `local_root`, which is the root of the collective but with a local coordinate system. The reason why

`local_id` can be of only one dimension is because the library works only on one dimension at a time.

Once these two variables have been defined, they have replaced their global counterparts in the collectives used.

Within the scope of this work, only the `reduce_fadds()`, `reduce_fmaxs()` and `gather()` collectives were adapted to follow this logic. The adaptation of the rest of the library is left for future work. It is worth noting that, due to the collaboration with the Cerebras staff to understand the inner workings of the `Collectives2D` library, the modified code was shared directly with their team. Whether this may result in an official update of the library in a future version of the SDK remains unknown.

3.2.6 Library infrastructure

For supporting the kernels, some infrastructure utilities have been defined, especially for connecting multiple kernels together and for sending exit signals to the whole WSE.

The library comes with 2 submodules that aid the user in the positioning of kernels and colors, which are respectively called `placer` and `drawer`. The `placer` module contains functions that allow the user to directly place a kernel given a starting coordinate and the parameters of the kernel. An example of function header is the following:

```
fn place_kernel(
    x_start: u16, y_start: u16,
    parameters: ...,
    colors: comptime_struct,
    tasks: comptime_struct,
    receive_from_custom_path: bool,
    last_kernel: bool) void { ... }
```

The kernel's parameters required change from kernel to kernel, but `colors`, `tasks`, `receive_from_custom_path` and `last_layer` are all common parameters. `colors` is a parameter of type struct, and contains all the colors needed for the kernel's inner working `tasks` is similar, but with all the tasks IDs that must be shared between PEs; `receive_from_custom_path` is a boolean flag that lets the kernel know that it will receive some data from a previous kernel, and not from the host; `last_kernel` denotes that the kernel is the last one on a chain and must be the one to send the exit signal.

There is one kind of kernel which is provided together with the library, that is used for filling the WSE: for a program to compile, all the PEs in the selected rectangular region must have received a code, but it may be possible that some PEs don't have to execute anything for the scope of an algorithm. For such cases, an empty kernel has been designed, which makes the PE simply wait for an exit signal from the last kernel. This kernel requires way less parameters, which are:

```
fn place_do_nothing(
    start_region: comptime_struct,
    end_region: comptime_struct,
    exit_color: color,
    exit_task: control_task_id) void { ... }
```

The `start_region` and `end_region` structs have two fields, `.x` and `.y`. This kind of struct is used to specify coordinates throughout the library. The `exit_task` parameter is needed because control wavelets can activate a task only if they carry its task ID, so the empty kernels will sync their exit task on said task ID.

The `drawer` module included has the purpose of placing colors, specifying as few parameters as possible, making it more comfortable to define routes. The module contains 3 functions:

- `draw_forward_path()`: this function draws a route for one color given starting and ending coordinates. It allows the user to specify after how many PEs it should bend vertically, forming an "*S*"-shaped path. An example of this function can be seen in Figure 3.1;
- `draw_branching_path()`: this function draws a branching route, which can reach simultaneously a variable number of PEs. It takes advantage of the colors property for which a color can receive from one single source, but send to more sources, broadcasting the signal. Similarly to `draw_forward_path()`, the user can specify where the path should start branching and expand vertically;
- `draw_exit_network()`: this function draws, starting from the PE that will send the exit signal, a color over all the rectangular region. It first draws two vertical routes, towards north and south, and then draws all the horizontal branches. For each PE, not only the color is set to send on the route, but also on the `RAMP`, so that a wavelet sent on this color will effectively be broadcasted to all PEs. A visual representation is available in Figure 3.1.

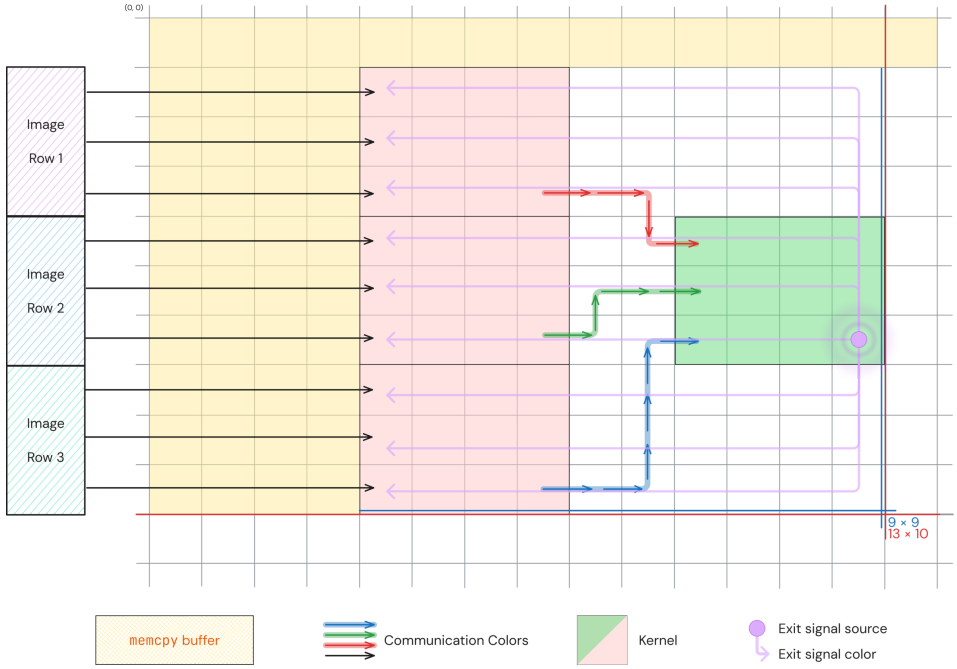

Figure 3.1. An example of a WSE region with 4 different kernels all connected between each other. The exit color is depicted in violet. Each PE over which the exit color flows through can receive the contents of the color (i.e. the color sends at each `RAMP` it encounters), except for the source of the exit signal, which exits independently after sending the message

Chapter 4
Evaluation of CereMath

While Chapter 3 focused on the implementation of the four proposed kernels, this chapter presents an analysis of their performance, alongside a discussion of the obtained results. The four kernels will be compared using the same sets of inputs across all platforms, allowing an easier comparison. To better explain the results, a common set of platform-agnostic metrics will be used.

4.1 Evaluation settings

To evaluate the proposed kernels, the following platforms will be used to run the kernels:

- for the WSE, the **Cerebras CS-3** systems of the Argonne National Laboratory (more precisely, these systems belong to the Argonne Leadership Computing Facility division) were used. An exhaustive description of the inner workings of the CS-3 systems and, in general, of the WSE-3 was provided in Chapter 2;

- for the GPU counterpart, an **NVIDIA A100 GPU** was used, from the Leonardo supercomputer, hosted by CINECA. On this cluster, the CUDA Toolkit 12.6 was used, which provides access to the `cuBLAS`, `cuRAND` and `cuDNN` libraries.

In these benchmarks, the weak scaling of the Cerebras programs is evaluated, alongside a comparison with the GPU version. All tests have been conducted with inputs that aimed to fill the WSE's SRAM for each PE as much as possible. For this reason, some inputs have reduced dimensions, but this remains coherent with the kernels' design choice: kernels should be compact and are intended to be used in synergy with one another.

Moreover, unlike the GEMV and GEMM kernels, Convolutions and Poolings typically handle images with smaller input sizes (for instance, the largest image size in the ResNet architecture is 224×224 [17]) and a large number of channels. For this reason, the inputs for the Convolution and Pooling kernels were chosen to match real-case scenarios.

To better evaluate the kernels, the amount of floating point operations per second (FLOP/s)[1] was computed for both the GPU and WSE versions. In both cases, we considered floating point operations using IEEE 754 32-bits precision numbers.

[1] Where only FLOP is shown, it will denote "floating point operation(s)"

4.1.1 Hardware metrics

To better compare the results, an overview of the metrics for the different hardware used is provided to establish a common ground for all evaluations.

First, the theoretical peak performance of the WSE and of the NVIDIA A100 is defined. As mentioned in Chapter 2, the WSE-3 operates at a frequency of 875 MHz and is capable of performing one FLOP per clock cycle. Given c as any amount of cores used for a computation, the theoretical peak performance is calculated as:

$$P_{\text{WSE}} = c \cdot 1\,\text{FLOP/cycle} \cdot 875\,\text{MHz} \tag{4.1}$$

In the following sections, this value will be more precisely computed, as the value of c is know after running the tests. Regarding the NVIDIA A100, NVIDIA states that the theoretical peak performance is 19.5 TFLOP/s when using 32-bits precision floating point numbers at full load. However, using its Tensor Cores, the theoretical performance is instead 156 TFLOP/s [25].

For each test on the WSE, 5 runs with the same set of parameters were performed; only the mean of these runs is reported, because in all cases the standard deviation was negligible ($\leq 3.7\%$). This decision stems from the fact that runs on the WSE are deterministic, since the SRAM access and computation times are deterministic too [18]. Moreover, this approach has also been adopted in other papers regarding software development for the WSE [24].

4.2 Kernels evaluation

In this section all the WSE kernels will be analyzed and compared to their respective GPU versions. All the plots shown report the timings for both the execution of the program and all `memcpy` operations (whether "host-to-device" or "device-to-host").

4.2.1 Convolution

In both the cases of the Convolution and Pooling kernels, the synthetic data used for the tests aimed to match a realistic data stream for a single convolution, with the except of the filter size, which was artifically augmented to demonstrate the kernel's performance and its limits.

All the runs were carried out with a linearly increasing width of $\mathbf{I}$ (with steps of 30 values each) and number of PEs (with steps of 6); the latter corresponds to the size of the filter $\mathbf{W}$. The overall performance of the kernel is shown in Figure 4.1, while the execution times of the runs are presented in Table 4.5.

Notably, the timing pattern performs a significant jump from the fifth test onwards. This is most likely due to the `reduce()` collective that must gather all the partial results; specifically, it appears to be a communication bottleneck. The source of the bottleneck is most likely the long path that the data must traverse to reach the root.

The number of operations performed by the Convolution kernel, based on the algorithmic description in Chapter 3, is detailed also in Table 4.1.

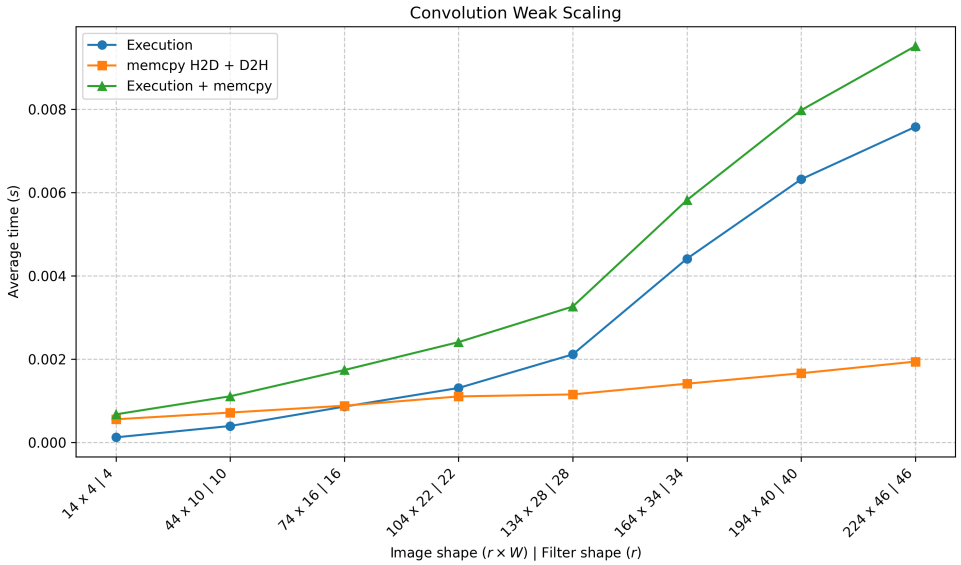

Figure 4.1. Results of the weak scaling test of the Convolution kernel

op0	op1	op2	op3	**Total**
/	$\mathbf{I}_{\text{part}} \odot W, \quad \forall\, C_{\text{out}}$ $\text{out} \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot f^2$ FLOP	`reduce()` $\text{out} \cdot C_{\text{out}}$ FLOP	$\mathbf{I}_{\text{part}} \odot W + \mathbf{b}$ $\text{out} \cdot C_{\text{out}}$ FLOP	/	$(\text{out} \cdot C_{\text{out}}) \cdot (C_{\text{in}} \cdot f^2 + 2)$

Table 4.1. FLOP performed by the WSE Convolution kernel

The total amount of operation (FLOP_{tot}) is thus given by Equation 4.2:

$$\begin{aligned} \text{FLOP}_{\text{tot}} &= \text{out} \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot f^2 + \text{out} \cdot C_{\text{out}} + \text{out} \cdot C_{\text{out}} = \\ &\text{out} \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot f^2 + 2 \cdot (\text{out} \cdot C_{\text{out}}) = (\text{out} \cdot C_{\text{out}}) \cdot (C_{\text{in}} \cdot f^2 + 2) \end{aligned} \tag{4.2}$$

The operational intensity for each test is listed in Table 4.5. In this kernel in particular, a large number of operations are performed (although they are not evenly distributed across the kernel). As a result, the amount of FLOP/s achieved is close the theoretical peak for the amount of PEs used. The variation in the growth of the FLOP/s across measurements is attributed to communication overhead.

4.2.2 Pooling

As mentioned in the evaluation of the Convolution kernel, the data used for the Pooling kernel is designed to match real-case scenarios. In this experiment, both the MaxPooling and AvgPooling kernels were evaluated using the same dataset. The increase in image width and the number of participating PEs is linear, with steps of 30 and 6, respectively. As shown in Figure 4.2, the two Pooling kernels exhibit similar timings.

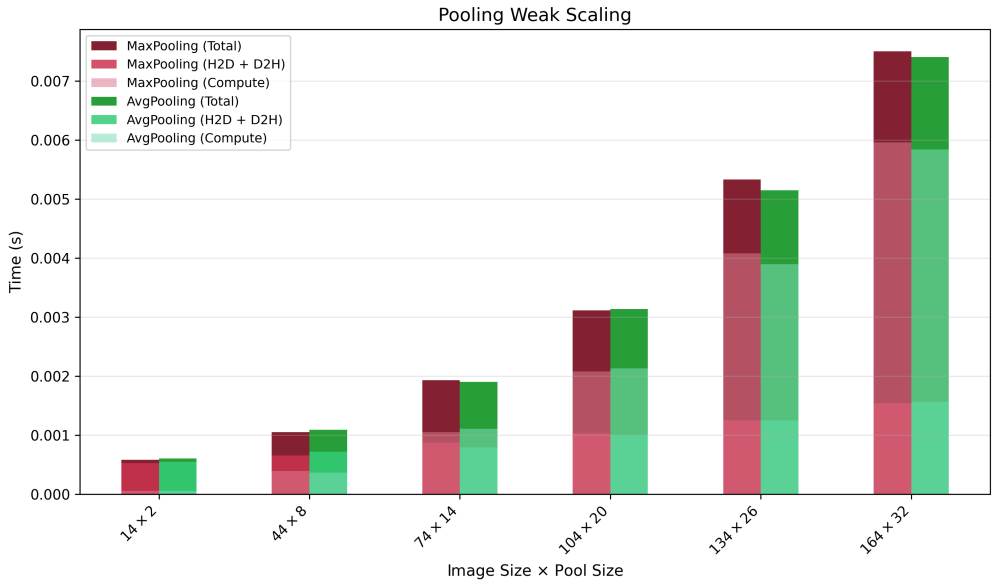

Figure 4.2. Results of the weak scaling test of the Pooling kernel, both MaxPooling (in red) and AvgPooling (in green)

According to Table 4.2, the AvgPooling kernel should perform more operations in op2, but this is not reflected in the timings. This is due to how the average operation is implemented: since it uses the DSDs built-in `@fmuls()`[2] with an outgoing DSD as destination, the WSE can perform the operation while moving data, resulting in zero additional cost.

The number of operations performed by the Pooling kernel is shown in Table 4.2

op0	op1	op2		op3	**Total**
/	$\max_{c \in C_{\text{in}}} \mathbf{I}_{\text{part}}$ or $\sum_{c}^{C_{\text{in}}} \mathbf{I}_{\text{part}}$ $\text{out} \cdot C_{\text{in}} \cdot f^2$ FLOP	`reduce()` $\text{out} \cdot C_{\text{in}}$ FLOP	+ Average $\text{out} \cdot C_{\text{in}}$ FLOP	/	$\text{out} \cdot C_{\text{in}} \cdot (f^2 + 2)$

Table 4.2. FLOP performed by the WSE Pooling kernel. The Average operation performed in op2 is performed only by the AvgPooling kernel

The total amount of operations is calculated using Equation 4.3:

$$\text{FLOP}_{\text{tot}} = \text{out} \cdot C_{\text{in}} \cdot f^2 + \text{out} \cdot C_{\text{in}} + \text{out} \cdot C_{\text{in}} = \text{out} \cdot C_{\text{in}} \cdot (f^2 + 1 + 1) \tag{4.3}$$

In the above equation, the additional operations performed by the AvgPooling kernel are shown in gray. The final amount of operations is used to compute the FLOP/s of each run, which ere reported in Table 4.6. The kernel presents a gap of 3 order of magnitude from the theoretical peak, meaning that the computational intensity can still be increased to take full advantage of the WSE comptuing capabilities. As with the Convolution kernel, it cannot be excluded that much of the collected time is generated by communication overhead.

[2]The multiplication is performed between the partial results and 1/part. res. size. The fraction is computed at compile time

4.2.3 GEMV

Regarding the GEMV kernel, tests were conducted with a very limited number of inputs. During the evaluation of the kernel, issues were encountered while running any configuration exceeding 16 rows of PEs, both on the SDK simulator and on the appliance. The most likely reason for this behaviour is a logic error in the adaptation of the `gather()` collective from the `Collectives2D` library.

Despite this issue, a brief formal analysis was conducted for the few collected runs. Since it is not possible to determine much from the timings alone (shown in Figure 4.3 and Table 4.7), the analysis will focus on the FLOP/s performed by the kernel.

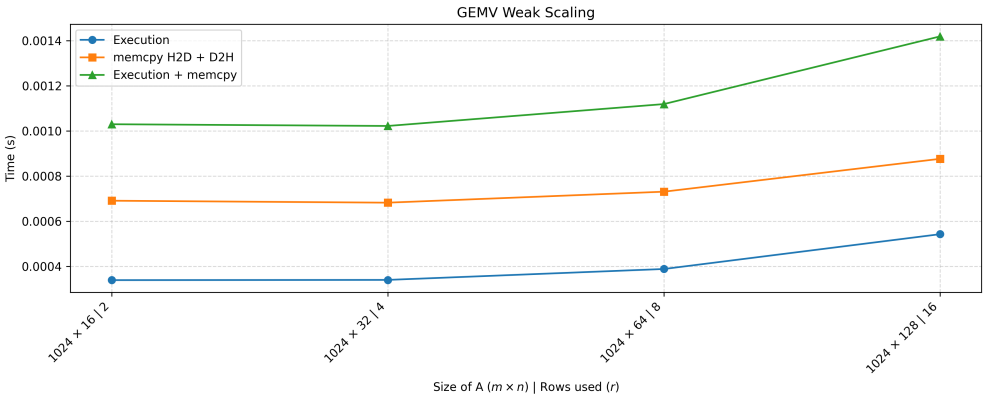

Figure 4.3. Results of the limited weak scaling test of the GEMV kernel

The number of FLOP performed by each sub-operation in the WSE GEMV kernel is described in Table 4.3:

op0	op1	op2		op3	**Total**
$\alpha \cdot \mathbf{A}$ $m \cdot n$ FLOP	$\alpha\mathbf{A} \times \mathbf{x}$ $m \cdot n$ FLOP	$\beta \cdot \mathbf{b}$ n FLOP	+	/ $\alpha\mathbf{Ax} + \beta\mathbf{b}$ n FLOP	$2n(m+1)$

Table 4.3. FLOP performed by the WSE GEMV kernel

Thus the total amount of operations performed is given by Equation 4.4:

$$\begin{aligned} \text{FLOP}_{\text{tot}} &= m \cdot n + m \cdot n + n + n = \\ &= n \cdot (2m + 2) = 2n \cdot (m + 1) \end{aligned} \tag{4.4}$$

Based on the result of the above equation, it is possible to compute the FLOPs performed by each run, which are also listed in Table 4.7. It can be noted that there is a difference of two orders of magnitude between the performance in FLOP/s of the different runs and the theoretical compute peak. This is most likely due to the communication overhead introduced by the collectives library.

4.2.4 GEMM

For the GEMM kernel, tests were conducted with much smaller inputs compared to all the other kernels. This is due to the kernel's high memory requirements, especially during the op2 stage, where a collective operation must be ran to gather all partial results.

The test was carried out with a linearly increasing input size for all three input matrices **A**, **B** and **C**, alongside a linear increase in the number of participating PEs. Each PE processes a constant amount of data (i.e., 8 rows). Above 11 PEs, the program fails to compile due to memory limitations.The results of the weak scaling test can be observed in Figure 4.4.

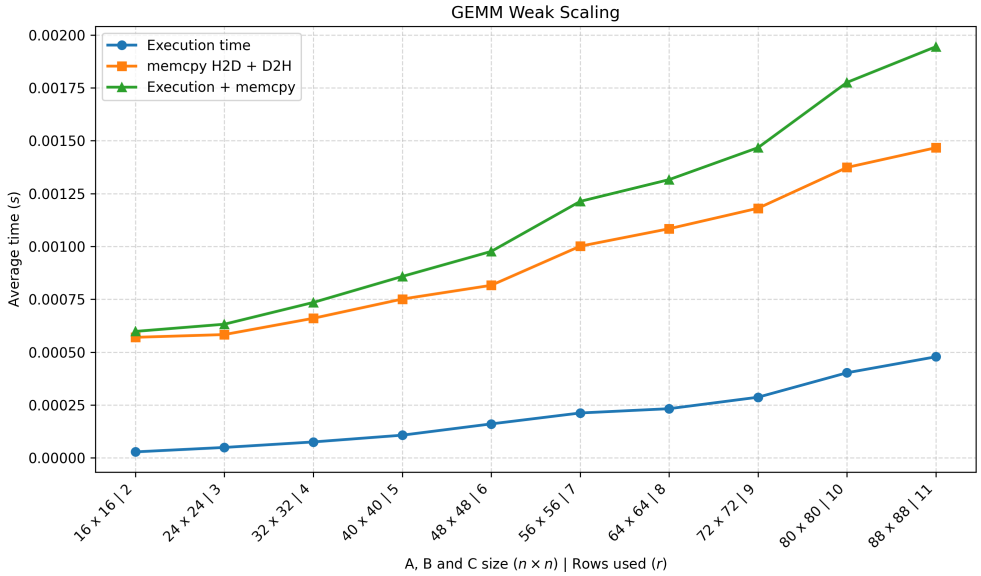

Figure 4.4. Results of the weak scaling test of the GEMM kernel

As can be seen from both the plot and the Table 4.8, the program's execution time keeps growing steadily with a similar rate, but the difference between the last runs in particular is notable. The suspected reason is as follows: in op2, the kernel must gather the partial results from all the PEs in the column, and, as the number of participating PEs increases, if the collective operation is not efficient enough, the communication overhead begins to emerge.

As detailed in Chapter 3, each sub-operation in the GEMM kernel computes a different operation, with the exception of op3. The computed operations are all listed in Table 4.4:

op0	op1	op2		op3	**Total**
$\alpha \cdot \mathbf{A}$ $m \cdot n$ FLOP	$\alpha\mathbf{A} \times \mathbf{B}$ $2(n \cdot m)$ FLOP	$\beta \cdot \mathbf{C}$ $m \cdot o$ FLOP	$+$ $\alpha\mathbf{AB} + \beta\mathbf{C}$ $m \cdot o$ FLOP	/	$m(3n + 2o)$

Table 4.4. FLOP performed by the WSE GEMM kernel

The total number of FLOP executed by the kernel is given by:

$$\begin{aligned} \text{FLOP}_{\text{tot}} = m \cdot n + 2(m \cdot n) + 2(m \cdot o) = \\ m \cdot (n + 2n + 2o) = m \cdot (3n + 2o) \end{aligned} \tag{4.5}$$

Table 4.8 reports the operational intensity of the kernel for each test, alongside the theoretical peak performance for the number of cores used. In all cases, the WSE's compute capabilities are heavily underutilized. A possible explanation is that nearly all of the data used by the kernel is not only processed to obtain a final result, but it's also transferred to an adjacent PE. For instance, in `op0` all the $m \cdot n$ values of matrix $\mathbf{A}$ are both multiplied by α (performing $m \cdot n$ operations) and also transmitted. Since moving data between adjacent PEs has a cost comparable to that of performing a single FLOP, completing `op0` requires approximately twice the time needed to strictly compute $\alpha\mathbf{A}$.

4.3 Comparison with GPU version

As a final step in this evaluation, a brief comparison will be made between the performance obtained on the WSE and the state-of-the-art libraries available for the NVIDIA GPUs. Specifically, the librares used for this comparison are `cuBLAS`, which provides the GEMV and GEMM functions, `cuRAND`, for initializing the data, and `cuDNN`, which provides both a Convolution and a Pooling function.

The tests performed on the GPU were run with the same parameters as the WSE, with the difference that the number of runs was increased from 5 to 30, and the standard deviation is not reported. This is because the GPU, by making use of local caches, has a non-deterministic behaviour. The results are reported in Table 4.9.

This comparison is conducted for the sole purpose of demonstrating the maturity of state-of-the-art libraries in developing tailored functions for key operations: hence, it's not expected to obtain results that differ by less than one order of magnitude. As can be seen from the FLOP/s results, for almost all the kernels the gap between the two versions is of 2 orders of magnitude, due to the issues previously highlighted for each kernel. While the WSE version of these kernels is approaching the production-ready libraries of the CUDA toolkit, much work remains be done to bridge this gap.

While comparing the two platforms however, it's important to stress that this behaviour is coherent with the design of the library: performance should be achieved through the concatenation of multiple kernels, or through the execution of multiple kernels in parallel. This mechanism is limited on a GPU, where it's instead preferred to saturate the threads and the Streaming Multiprocessors. For achieving the same effect on a traditional architecture, while keeping a large scale, one would need to use a series of interconnected nodes; this not only adds communication overhead, but also a series of non-trivial networking and coordination challenges between the workers. The WSE removes, on domains that fit within the PE's grid size, the need of a network and allows the user to focus entirely on how to move the data and how to process them.

I_W	PE rows (f)	Init time	H2D time	Exec time	D2H time	Total time	FLOP/s	P_{WSE} (FLOP/s)
14	4	$81.78s$	$3.69 \cdot 10^{-4}s$	$1.19 \cdot 10^{-4}s$	$1.83 \cdot 10^{-4}s$	$81.78s$	$1.609 \cdot 10^{8}$	$1.4 \cdot 10^{10}$
44	10	$83.15s$	$4.47 \cdot 10^{-4}s$	$3.92 \cdot 10^{-4}s$	$2.67 \cdot 10^{-4}s$	$83.15s$	$8.887 \cdot 10^{8}$	$3.5 \cdot 10^{10}$
74	16	$84.43s$	$6.09 \cdot 10^{-4}s$	$8.57 \cdot 10^{-4}s$	$2.70 \cdot 10^{-4}s$	$84.43s$	$1.724 \cdot 10^{9}$	$5.6 \cdot 10^{10}$
104	22	$89.71s$	$6.95 \cdot 10^{-4}s$	$1.31 \cdot 10^{-3}s$	$4.07 \cdot 10^{-4}s$	$89.71s$	$2.994 \cdot 10^{9}$	$7.7 \cdot 10^{10}$
134	28	$91.02s$	$7.06 \cdot 10^{-4}s$	$2.11 \cdot 10^{-3}s$	$4.45 \cdot 10^{-4}s$	$91.03s$	$3.851 \cdot 10^{9}$	$9.8 \cdot 10^{10}$
164	34	$92.26s$	$8.35 \cdot 10^{-4}s$	$4.41 \cdot 10^{-3}s$	$5.73 \cdot 10^{-4}s$	$92.27s$	$3.322 \cdot 10^{9}$	$1.2 \cdot 10^{11}$
194	40	$93.87s$	$1.06 \cdot 10^{-3}s$	$6.32 \cdot 10^{-3}s$	$5.99 \cdot 10^{-4}s$	$93.88s$	$3.792 \cdot 10^{9}$	$1.4 \cdot 10^{11}$
224	46	$94.82s$	$1.26 \cdot 10^{-3}s$	$7.58 \cdot 10^{-3}s$	$6.81 \cdot 10^{-4}s$	$94.83s$	$4.825 \cdot 10^{9}$	$1.6 \cdot 10^{11}$

Table 4.5. Convolution weak scaling results summary. In this test, C_{in} and C_{out} are both constant and fixed at, respectively, 3 and 64

Pool type	I_W	PE rows (f)	C_{in}	Init Time	H2D Time	Exec Time	D2H Time	Total Time	FLOP/s	P_{WSE} (FLOP/s)
MaxPooling	14	2	64	$80.84s$	$3.13 \cdot 10^{4}$	$5.90 \cdot 10^{5}$	$2.10 \cdot 10^{4}$	$80.84s$	$3.796 \cdot 10^{7}$	$7.0 \cdot 10^{9}$
	44	8	64	$82.97s$	$4.22 \cdot 10^{4}$	$3.93 \cdot 10^{4}$	$2.36 \cdot 10^{4}$	$82.97s$	$2.011 \cdot 10^{8}$	$2.8 \cdot 10^{10}$
	74	14	64	$84.00s$	$5.46 \cdot 10^{4}$	$1.05 \cdot 10^{3}$	$3.31 \cdot 10^{4}$	$84.00s$	$3.707 \cdot 10^{8}$	$4.9 \cdot 10^{10}$
	104	20	64	$85.20s$	$6.85 \cdot 10^{4}$	$2.08 \cdot 10^{3}$	$3.51 \cdot 10^{4}$	$85.20s$	$5.308 \cdot 10^{8}$	$7.0 \cdot 10^{10}$
	134	26	64	$90.95s$	$7.78 \cdot 10^{4}$	$4.08 \cdot 10^{3}$	$4.75 \cdot 10^{4}$	$90.96s$	$5.843 \cdot 10^{8}$	$9.1 \cdot 10^{10}$
	164	32	64	$92.24s$	$9.73 \cdot 10^{4}$	$5.96 \cdot 10^{3}$	$5.67 \cdot 10^{4}$	$92.35s$	$7.378 \cdot 10^{8}$	$1.1 \cdot 10^{11}$
AvgPooling	14	2	64	$80.67s$	$3.28 \cdot 10^{4}$	$5.90 \cdot 10^{5}$	$2.19 \cdot 10^{4}$	$80.67s$	$4.556 \cdot 10^{7}$	$7.0 \cdot 10^{9}$
	44	8	64	$82.97s$	$4.59 \cdot 10^{4}$	$3.68 \cdot 10^{4}$	$2.64 \cdot 10^{4}$	$82.98s$	$2.181 \cdot 10^{8}$	$2.8 \cdot 10^{10}$
	74	14	64	$84.30s$	$5.10 \cdot 10^{4}$	$1.11 \cdot 10^{3}$	$2.86 \cdot 10^{4}$	$84.30s$	$3.544 \cdot 10^{8}$	$4.9 \cdot 10^{10}$
	104	20	64	$85.39s$	$6.10 \cdot 10^{4}$	$2.13 \cdot 10^{3}$	$3.97 \cdot 10^{4}$	$85.39s$	$5.189 \cdot 10^{8}$	$7.0 \cdot 10^{10}$
	134	26	64	$90.43s$	$7.72 \cdot 10^{4}$	$3.90 \cdot 10^{3}$	$4.77 \cdot 10^{4}$	$90.43s$	$6.125 \cdot 10^{8}$	$9.1 \cdot 10^{10}$
	164	32	64	$92.29s$	$1.01 \cdot 10^{3}$	$5.84 \cdot 10^{3}$	$5.54 \cdot 10^{4}$	$92.29s$	$7.535 \cdot 10^{8}$	$1.1 \cdot 10^{11}$

Table 4.6. Pooling weak scaling results summary

n	m	r	PE rows	Init time	H2D time	Exec time	D2H time	Total time	FLOP/s	P_{WSE} (FLOP/s)
1024	16	8	2	81.43	$5.2433 \cdot 10^{-4}s$	$3.3900 \cdot 10^{-4}s$	$1.6667 \cdot 10^{-4}s$	$81.42s$	$1.027 \cdot 10^{8}$	$7.0 \cdot 10^{9}$
1024	32	8	4	82.38	$5.1867 \cdot 10^{-4}s$	$3.4000 \cdot 10^{-4}s$	$1.6367 \cdot 10^{-4}s$	$82.38s$	$1.988 \cdot 10^{8}$	$1.4 \cdot 10^{10}$
1024	64	8	8	84.01	$5.5300 \cdot 10^{-4}s$	$3.8833 \cdot 10^{-4}s$	$1.7800 \cdot 10^{-4}s$	$84.00s$	$3.428 \cdot 10^{8}$	$2.8 \cdot 10^{10}$
1024	128	8	16	85.50	$6.9500 \cdot 10^{-4}s$	$5.4267 \cdot 10^{-4}s$	$1.8167 \cdot 10^{-4}s$	$85.50s$	$4.868 \cdot 10^{8}$	$5.6 \cdot 10^{10}$

Table 4.7. GEMV weak scaling results summary

m	n	o	r	PE rows	Init time	H2D time	Exec time	D2H time	Total time	FLOP/s	P_{WSE} (FLOP/s)
16	16	16	8	2	$81.77s$	$4.0 \cdot 10^{-4}s$	$2.8 \cdot 10^{-5}s$	$1.7 \cdot 10^{-4}s$	$81.77s$	$4.571 \cdot 10^{7}$	$7.0 \cdot 10^{9}$
24	24	24	8	3	$82.98s$	$4.0 \cdot 10^{-4}s$	$4.9 \cdot 10^{-5}s$	$1.9 \cdot 10^{-4}s$	$82.98s$	$5.877 \cdot 10^{7}$	$1.0 \cdot 10^{10}$
32	32	32	8	4	$83.33s$	$4.3 \cdot 10^{-4}s$	$7.5 \cdot 10^{-5}s$	$2.3 \cdot 10^{-4}s$	$83.33s$	$6.826 \cdot 10^{7}$	$1.4 \cdot 10^{10}$
40	40	40	8	5	$83.41s$	$4.5 \cdot 10^{-4}s$	$1.1 \cdot 10^{-4}s$	$3.1 \cdot 10^{-4}s$	$83.41s$	$7.476 \cdot 10^{7}$	$1.7 \cdot 10^{10}$
48	48	48	8	6	$84.07s$	$4.8 \cdot 10^{-4}s$	$1.6 \cdot 10^{-4}s$	$3.3 \cdot 10^{-4}s$	$84.07s$	$7.185 \cdot 10^{7}$	$2.1 \cdot 10^{10}$
56	56	56	8	7	$84.34s$	$6.0 \cdot 10^{-4}s$	$2.1 \cdot 10^{-4}s$	$4.0 \cdot 10^{-4}s$	$84.34s$	$7.396 \cdot 10^{7}$	$2.4 \cdot 10^{10}$
64	64	64	8	8	$83.50s$	$5.6 \cdot 10^{-4}s$	$2.3 \cdot 10^{-4}s$	$5.3 \cdot 10^{-4}s$	$83.51s$	$8.815 \cdot 10^{7}$	$2.8 \cdot 10^{10}$
72	72	72	8	9	$83.77s$	$5.7 \cdot 10^{-4}s$	$2.9 \cdot 10^{-4}s$	$6.1 \cdot 10^{-4}s$	$83.77s$	$9.042 \cdot 10^{7}$	$3.1 \cdot 10^{10}$
80	80	80	8	10	$83.88s$	$6.4 \cdot 10^{-4}s$	$4.0 \cdot 10^{-4}s$	$7.3 \cdot 10^{-4}s$	$83.89s$	$7.954 \cdot 10^{7}$	$3.5 \cdot 10^{10}$
88	88	88	8	11	$83.95s$	$6.5 \cdot 10^{-4}s$	$4.8 \cdot 10^{-4}s$	$8.2 \cdot 10^{-4}s$	$83.95s$	$8.100 \cdot 10^{7}$	$3.8 \cdot 10^{10}$

Table 4.8. GEMM weak scaling results summary

Test	Data size	Execution time	FLOP/s (GPU)	FLOP/s (WSE)
Convolution	14×4	$1.91 \cdot 10^{-5}s \pm 0.0000s$	$1.01 \cdot 10^{9} \pm 0.43$	$1.609 \cdot 10^{8}$
	44×10	$1.71 \cdot 10^{-5}s \pm 0.0000s$	$8.90 \cdot 10^{9} \pm 0.31$	$8.887 \cdot 10^{8}$
	74×16	$1.88 \cdot 10^{-5}s \pm 0.0000s$	$2.18 \cdot 10^{10} \pm 0.56$	$1.724 \cdot 10^{9}$
	104×22	$2.04 \cdot 10^{-5}s \pm 0.0000s$	$3.88 \cdot 10^{10} \pm 0.79$	$2.994 \cdot 10^{9}$
	134×28	$2.16 \cdot 10^{-5}s \pm 0.0000s$	$6.01 \cdot 10^{10} \pm 0.14$	$3.851 \cdot 10^{9}$
	164×34	$2.41 \cdot 10^{-5}s \pm 0.0000s$	$8.03 \cdot 10^{10} \pm 0.51$	$3.322 \cdot 10^{9}$
	194×40	$2.58 \cdot 10^{-5}s \pm 0.0000s$	$1.04 \cdot 10^{11} \pm 0.57$	$3.792 \cdot 10^{9}$
	224×46	$2.87 \cdot 10^{-5}s \pm 0.0000s$	$1.24 \cdot 10^{11} \pm 0.50$	$4.825 \cdot 10^{9}$
MaxPooling	14×2	$6.76 \cdot 10^{-6}s \pm 0.0000s$	$0.27 \cdot 10^{9} \pm 0.02$	$3.796 \cdot 10^{7}$
	44×8	$1.32 \cdot 10^{-5}s \pm 0.0000s$	$1.89 \cdot 10^{9} \pm 0.68$	$2.011 \cdot 10^{8}$
	74×14	$1.39 \cdot 10^{-5}s \pm 0.0000s$	$5.14 \cdot 10^{9} \pm 1.65$	$3.707 \cdot 10^{8}$
	104×20	$1.26 \cdot 10^{-5}s \pm 0.0000s$	$1.17 \cdot 10^{10} \pm 0.39$	$5.308 \cdot 10^{8}$
	134×26	$1.31 \cdot 10^{-5}s \pm 0.0000s$	$1.91 \cdot 10^{10} \pm 0.66$	$5.843 \cdot 10^{8}$
	164×32	$8.82 \cdot 10^{-6}s \pm 0.0000s$	$3.81 \cdot 10^{10} \pm 0.13$	$7.378 \cdot 10^{8}$
AvgPooling	14×2	$6.89 \cdot 10^{-6}s \pm 0.0000s$	$0.26 \cdot 10^{9} \pm 0.02$	$4.556 \cdot 10^{7}$
	44×8	$1.33 \cdot 10^{-5}s \pm 0.0000s$	$1.86 \cdot 10^{9} \pm 0.64$	$2.181 \cdot 10^{8}$
	74×14	$1.39 \cdot 10^{-5}s \pm 0.0000s$	$5.18 \cdot 10^{9} \pm 1.70$	$3.544 \cdot 10^{8}$
	104×20	$1.48 \cdot 10^{-5}s \pm 0.0000s$	$9.55 \cdot 10^{9} \pm 2.89$	$5.189 \cdot 10^{8}$
	134×26	$1.38 \cdot 10^{-5}s \pm 0.0000s$	$1.79 \cdot 10^{10} \pm 0.64$	$6.125 \cdot 10^{8}$
	164×32	$9.02 \cdot 10^{-6}s \pm 0.0000s$	$3.73 \cdot 10^{10} \pm 0.15$	$7.535 \cdot 10^{8}$
GEMV	1024×16	$0.0083s \pm 0.0000s$	$3.93 \cdot 10^{9} \pm 0.02$	$1.027 \cdot 10^{8}$
	1024×32	$0.0083s \pm 0.0001s$	$3.93 \cdot 10^{9} \pm 0.06$	$1.988 \cdot 10^{8}$
	1024×64	$0.0083s \pm 0.0000s$	$3.93 \cdot 10^{9} \pm 0.08$	$3.428 \cdot 10^{8}$
	1024×128	$0.0083s \pm 0.0001s$	$3.93 \cdot 10^{9} \pm 0.22$	$4.868 \cdot 10^{8}$
GEMM	$16 \times 16 \times 16$	$0.0056s \pm 0.0000s$	$1.47 \cdot 10^{9} \pm 0.01$	$4.571 \cdot 10^{7}$
	$24 \times 24 \times 24$	$0.0062s \pm 0.0000s$	$4.49 \cdot 10^{9} \pm 0.02$	$5.877 \cdot 10^{7}$
	$32 \times 32 \times 32$	$0.0062s \pm 0.0000s$	$1.06 \cdot 10^{10} \pm 0.07$	$6.826 \cdot 10^{7}$
	$40 \times 40 \times 40$	$0.0073s \pm 0.0000s$	$1.75 \cdot 10^{10} \pm 0.07$	$7.476 \cdot 10^{7}$
	$48 \times 48 \times 48$	$0.0073s \pm 0.0000s$	$3.01 \cdot 10^{10} \pm 0.11$	$7.185 \cdot 10^{7}$
	$56 \times 56 \times 56$	$0.0075s \pm 0.0000s$	$4.67 \cdot 10^{10} \pm 0.18$	$7.396 \cdot 10^{7}$
	$64 \times 64 \times 64$	$0.0075s \pm 0.0000s$	$7.00 \cdot 10^{10} \pm 0.23$	$8.815 \cdot 10^{7}$
	$72 \times 72 \times 72$	$0.0082s \pm 0.0000s$	$9.12 \cdot 10^{10} \pm 0.47$	$9.042 \cdot 10^{7}$
	$80 \times 80 \times 80$	$0.0082s \pm 0.0000s$	$1.25 \cdot 10^{11} \pm 0.52$	$7.954 \cdot 10^{7}$
	$88 \times 88 \times 88$	$0.0082s \pm 0.0000s$	$1.65 \cdot 10^{11} \pm 0.77$	$8.100 \cdot 10^{7}$

Table 4.9. Differences between the performances of the WSE and GPU kernels

Chapter 5
Conclusions

Having presented the CereMath's kernels from an algorithmic and a performance perspective, it is possible to draw some conclusions whether this project may see a viable future or not, if so, what further work can be done to enhance it.

5.1 Validity of the project and future work

As described in Section 4.3, the gap between the proposed version of CereMath and libraries such as `cuBLAS` and `cuDNN`, which is about 2 orders of magnitude, is still too large. Several issues have been identified within the kernels; these are summarized below, along with considerations on how to address them to achieve better performance:

- **communication**: a common problem across all kernels is that a significant amount of time is lost during the execution of collective functions belonging to the `Collectives2D` library provided by Cerebras. Although such library is useful for small programs, it presents several issues that make it unsuitable for even medium-sized kernels (i.e., kernels with more than 16 PEs along a single collective). This can be mitigated by using external libraries or custom sets of collectives, such as the one proposed by P. Luczynski et al. [24] for the WSE-2;
- **spreading the workload**: especially in the Convolution and Pooling kernels, the majority of the computational workload is concentrated in `op1` and `op2`, effectively leaving 50% of the PEs to act as buffers. While the proposed separation appears logical, spreading the workload across all of the kernel's surface would allow to achieve a better performance overall;
- **data streaming**: although not utilized in this work, data streaming is a feature that could significantly help alleviate memory consumption. By leveraging this feature, the core challenge shifts from efficient memory utilization to sustaining data flow and implementing a continuous pipeline.

Many of the issues encountered while developing this library stemmed from the fact that many details regarding the WSE-3 and its inner workings remain undisclosed. This is because Cerebras is still a new (and small) company and prefers to keep such information condifential. Although it is possible to gather more information and interact directly with company employees through the developers' forum, this heavily slows down the development and debugging phases.

On the software side, there are several other solutions that aim to make writing CSL code a less tedious experience: some examples are the SCALA framework [14] and the PyFlame project [5]. The absence of a Language Server Protocol (LSP) is

also notably felt. However, independent projects external from Cerebras, such as the implementation of a custom LSP, not only help the users who are already developing software for this architecture, but also allow the company and their product to gain popularity, potentially motivating the release of more user-friendly tools.

Finally, after evaluating the entire library, it is compelling to answer to the following question:

"*Does it make sense to have a unified set of BLAS-like, general purpose kernels for the WSE?*"

As mentioned in Chapter 3, dataflow architectures allow developers to create programs built around the data, rather than the other way around; by creating a library of fixed kernels, the opposite effect would seem be achieved. Nevertheless, the answer is still considered to be "*yes, it makes sense*" for the following reasons:

- first, while a library can follow the directives of a single standard, multiple versions can exist to accommodate different use cases (consider the multiple platform-specific versions available for different CPUs and GPUs). This can be seen also with other popular standards, that have multiple implementations, such as MPI for node communication and OpenFlow for network programming. By adhering to a standard, the corresponding implementations will perform similarly, only with a more fine-grained control over specific elements such as hardware resources;

- secondly, a library allows developers to contribute with enhancements and optimizations, allowing any project based on it to take advantage of the work of many. Moreover, it may encourage collaboration with Cerebras employees, allowing for the writing of code that can better exploit the underlying hardware.

While this work is not ready for use in larger projects, it aimes to serve as a starting point for the creation of common resources, or perhaps to spark interest among fellow developers who see potential for success in this idea. What this work can also be proud of is to have helped understanding a design issue in the `Collectives2D` library, providing a simple yet (possibly) effective way for fixing said issue.

5.2 The future of the WSE and dataflow architectures in ML/HPC settings

In this work, the Cerebras WSE has been treated as a solid architecture, akin to a GPGPU, without delving too deeply into the practical advantages of using this platform over a traditional cluster. Should companies increasingly opt for the WSE, or should they stick to traditional, highly optimized architectures?

A reason companies might prefer classical architectures like GPUs is the mature ecosystem that supports these devices. While the Cerebras community is attracting increasing interest from new users, the gap remains substantial; overall it's more cost-effective to use well-supported hardware rather than a than a novel architecture. This is further enhanced by the fact that a CS-3 system costs in the order of millions

of USD, while GPUs are in the order of thousands. However, there are advantages to using a new architecture, such as the market diversification. Currently, the accelerator market is dominated by AMD and NVIDIA; introducing a new competitor enhances competition, which ultimately benefits all interested consumers. Supporting Cerebras would enhance market growth and potentially drive the release of more advanced features across the industry.

Overall, the WSE is no stranger to applications outside the AI sphere. Given the increasing popularity of this architecture, it wouldn't be surprising to see it applied to other fields such as fluid dynamics, molecular dynamics [26] or environmental simulations. The primary barrier to the adoption of the WSE and similar dataflow architecture is that existing code must be rewritten and adapted to achieve good performances, which is a non-trivial task. This may change over time as better frameworks emerge, but bridging the gap between traditional and domain-specific architectures will require significant time and effort from the community. Until then, much work remains to be done and this thesis, although ambitious, aimed to contribute to this exact effort. Future research may eventually build upon the foundation laid here and succeed in establishing a common standard for these essential building blocks. Whether the case will be and even if the results did not fully meet initial expectations, this work can hopefully be the first step.

ACKNOWLEDGEMENTS

Fra tutti i capitoli della tesi, credo che questo sia il più complesso da scrivere: non perché sia difficile trovare qualcuno da ringraziare (forse le persone sono un po' troppe), ma perché non so proprio come farlo nel modo giusto, per tutte quelle persone che mi hanno accompagnato in questi 3 anni di triennale. E forse per molti di quelli che verranno menzionati sarà pure abbastanza un "*ringrazio x*", ma da parte mia non riesco a convincermene.

"*Le parole sono potenti veicoli di emozioni*". O almeno, così sarebbe giustificabile un qualsiasi ringraziamento su queste pagine. Ma per tutti voi che leggerete, sappiate che ogni ringraziamento che troverete qui scritto ha un profondo significato per me: se il vostro nome compare in queste righe, vuol dire che avete veramente impattato la mia vita in modo positivo, e che questo mio traguardo lo voglio condividere con tutti voi.

Prima di tutti, voglio ringraziare la mia famiglia: mio padre Carlo, che mi ha trasmesso fin da piccolo una grande passione per i computer e per l'informatica; mia madre Raffaella, che mi ha sempre aiutato in momenti in cui ho dubitato di me stesso e mi ha convinto a scegliere questo corso di laurea; mia sorella Eleonora, con cui ho stretto un legame maggiore durante questi anni, e che mi ha sempre fatto sorridere con le battute più assurde; i miei nonni Giovanni, Mario e Mirella, con i quali ho avuto modo di scambiare idee e pensieri, più di quanto abbia mai potuto fare, garantendomi un punto di vista alternativo su molte questioni di vario genere; i miei zii Ilaria, Massimiliano, Daniela, Ileana, Paola e Sabatino, per esserci sempre stati, e per il rapporto più stretto che si sta stringendo in questi anni (chissà, magari è veramente l'età). A loro tutti devo la riuscita di questo percorso, e di questa tesi. Poi chissà, magari non avrete capito molto di quello che ho scritto nei precedenti capitoli, ma almeno spero che le immagini vi siano piaciute!

A Sara, la persona che più di tutte si è dovuta sorbire le mie interminabili lamentele sulla Sapienza e su come non ce l'avrei mai fatta, rassicurandomi che in un modo o nell'altro ce l'avremmo fatta entrambi. In questi anni assieme siamo riusciti a costruire un rapporto che all'inizio mi sarei solamente sognato, mantenendo una relazione a distanza da sogno. Senza il suo appoggio, le sue risate, le sue cene a base di patate, i suoi racconti sulle pecorelle in Scozia e le sere a disperarci su It Takes Two, Genshin e Overcooked difficilmente avrei superato serenamente questo percorso. E per quanto come compagno di vita possa fare di meglio, spero veramente di riuscire a far restare quel sorriso, che valorizza ogni momento speso insieme.

A tutti i miei amici, conosciuti o non durante questi tre anni e mezzo, a tutte quelle persone che hanno riempito la mia vita giorno dopo giorno: senza di voi questi 3 anni non sarebbero passati così leggermente. Ora, per menzionarvi, permettetemi di fare una stupidaggine da buon informatico: siete tutti importanti per me, ma non voglio che l'ordine di menzione possa essere ricondotto a una preferenza da parte mia. Sappiate dunque che l'ordine in cui vi nominerò sarà deciso da un simpaticissimo `np.random.shuffle()` (che, per coloro che non conoscono NumPy, è essenzialmente una funzione per mischiare una lista in modo casuale). Detto questo, diamo un occhio alla lista.

Al gruppo delle bimbe, per le serate assieme, i pranzi a mensa, le ore passate assieme a lezione e quei viaggi nel disagio completo: siete stati il centro della mia vita qui a Roma, mi avete insegnato e trasmesso tanto (come l'interesse per la mobilità pubblica, ormai sono stato condizionato). Non sarò stato forse il miglior amico che avreste potuto avere, ma per me siete il meglio che potessi desiderare. A Leila, Elia, Oscar, Andrea, Lorenzo, Francesco C., Giovanni, Francesco DB, Maria, Alessandro, Helia, Maria Q. e Davide: grazie per questi 3 anni, e a cento di questi giorni!

A SapienzaStudents, per avermi accolto in un progetto nonostante non mi conoscesse nessuno, per la fiducia nei miei confronti, e per avermi insegnato a essere una componente più attiva della vita studentesca. A Matteo, Lorenzo, Robert e il resto dello staff, per avermi introdotto al servizio per gli altri.

Al gruppo del bunker, per le uscite matte a San Lorenzo il venerdì in sessione, per le giornate di studio assieme, per le innumerevoli pause al bar, e per tutti i momenti passati assieme nel disagio. A Benedetta, Giuseppe, Zosia, Jacopo R., Mattia, Anna, Dana, Lorenza, Roberto, Jay, Vincenzo, Samuele, Claudia, Jacopo M. e Nadia, gli spritz del venerdì a 3,50 € sono difficili da dimenticare.

A PlumJuice, per l'IndySCC25 e le competizioni future, per avermi ispirato al mondo dell'HPC, e per i mille progetti portati avanti nel quasi inesistente tempo libero che avevo. Ad Andrea, Elia, Alessandro, Lorenzo, Francesco "Ciz", Saverio, Maria, Leila, Luca e Francesco DB, affinché possiamo sempre continuare a guardare lontano, e a essere un team coeso.

A Intercultura e al centro locale di Roma Est, per avermi fatto sentir parte di una grande rete di volontari, per aver potuto dare una mano in un progetto dal grande impatto sociale, e per avermi accolto come uno di loro fin dal primo giorno. A tutto lo staff di Colle Val d'Elsa, ad Alessio, Chiara, Andrea, Chloe, Nicole, Lorenzo e tutti gli altri volontari del centro, per avermi fatto crescere come formatore ed essersi fidati nei 2 anni di responsabile invio.

Al dipartimento di informatica della Sapienza, a quei professori che mi hanno fatto appassionare al mio percorso, che non hanno mostrato il lato pesante del mondo accademico, e che hanno sempre agito per gli studenti. A voi va un sentito ringraziamento, e la speranza che questo dipartimento possa continuare a contare su persone come voi. Soprattutto, grazie al prof. Daniele De Sensi, per la sua enorme disponibilità, e per avermi motivato a esplorare il mondo dell'High Performance Computing.

Grazie, **a tutte e tutti**. Che questo momento possa motivare e incentivare nei momenti più bassi, come memento che con abbastanza forza di volontà i nostri sogni sono raggiungibili, e che c'è sempre una luce in fondo al tunnel. Burn your dread, always move forward.

[1] *Dataset of publicly released ML models, epoch.ai*. https://epoch.ai/data/ai-models?enableTopN=true&topN=5&cameraX0=1990-4-3&cameraY0=10034.484963183682&cameraX1=2026-4-12&cameraY1=5.00000000000004e%2B26#explore-the-data.

[2] *Efficient Computer homepage*. https://web.archive.org/web/20251029150914/https://www.efficient.computer/.

[3] *MIT Dataflow Research Project Homepage*. https://web.archive.org/web/20120730230237/http://cnc.cs.manchester.ac.uk/projects/dataflow.html.

[4] *IEEE Std 754-2019 (Revision of IEEE Std 754-2008) IEEE Standard for Floating-Point Arithmetic*, (2019).

[5] *CTO92/PyFlame: A Python Deep Learning framework with lazy evaluation, automatic differentiation, and a PyTorch-like API*. https://web.archive.org/web/20260313142013/https://github.com/CTO92/PyFlame, Mar. 2026.

[6] L. BLACKFORD, J. DEMMEL, J. DONGARRA, I. DUFF, S. HAMMARLING, G. HENRY, M. HEROUX, L. KAUFMAN, A. LUMSDAINE, A. PETITET, R. POZO, K. REMINGTON, AND R. WHALEY, *An updated set of Basic Linear Algebra Subprograms (BLAS)*, ACM Transactions on Mathematical Software, 28 (2002), pp. 135–151.

[7] CEREBRAS COMPANY, *Cerebras - WSE Product Page*. https://www.cerebras.ai/chip.

[8] ——, *Cerebras SDK documentation*. https://sdk.cerebras.net.

[9] ——, *Cerebras System 3 Datasheet*.

[10] ——, *Cerebras Wafer Scale Cluster Datasheet*.

[11] J. J. DONGARRA, J. DU CROZ, S. HAMMARLING, AND I. S. DUFF, *A Set of Level 3 Basic Linear Algebra Subprograms*, ACM Trans. Math. Softw., 16 (1990), pp. 1–17.

[12] J. J. DONGARRA, J. DU CROZ, S. HAMMARLING, AND R. J. HANSON, *An Extended Set of FORTRAN Basic Linear Algebra Subprograms*, ACM Trans. Math. Softw., 14 (1988), pp. 1–17.

[13] A. DOSOVITSKIY, L. BEYER, A. KOLESNIKOV, D. WEISSENBORN, X. ZHAI, T. UNTERTHINER, M. DEHGHANI, M. MINDERER, G. HEIGOLD, S. GELLY, J. USZKOREIT, AND N. HOULSBY, *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, June 2021.

[14] L. GIANINAZZI, T. BEN-NUN, AND T. HOEFLER, *SPADA: A Spatial Dataflow Architecture Programming Language*, Nov. 2025.

[15] GOOGLE, *Introduction to Cloud TPU*. https://docs.cloud.google.com/tpu/docs/intro-to-tpu.

[16] C. HE, Y. HUANG, P. MU, Z. MIAO, J. XUE, L. MA, F. YANG, AND L. MAI, *WaferLLM: Large Language Model Inference at Wafer Scale*.

[17] K. HE, X. ZHANG, S. REN, AND J. SUN, *Deep Residual Learning for Image Recognition*.

[18] T. HOEFLER AND R. BELLI, *Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results*, in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '15, New York, NY, USA, Nov. 2015, Association for Computing Machinery, pp. 1–12.

[19] M. JAMES, M. TOM, P. GROENEVELD, AND V. KIBARDIN, *ISPD 2020 Physical Mapping of Neural Networks on a Wafer-Scale Deep Learning Accelerator*, in Proceedings of the 2020 International Symposium on Physical Design, Taipei Taiwan, Mar. 2020, ACM, pp. 145–149.

[20] A. KRIZHEVSKY, I. SUTSKEVER, AND G. E. HINTON, *ImageNet classification with deep convolutional neural networks*, in Advances in Neural Information Processing Systems, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, eds., vol. 25, Curran Associates, Inc., 2012.

[21] C. L. LAWSON, R. J. HANSON, D. R. KINCAID, AND F. T. KROGH, *Basic linear algebra subprograms for fortran usage*, ACM Trans. Math. Softw., 5 (1979), pp. 308–323.

[22] Y. LECUN, L. BOTTOU, Y. BENGIO, AND P. HAFFNER, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 86 (1998), pp. 2278–2324.

[23] S. LIE, *The Cerebras Wafer-Scale Architecture for Deep Learning*, Oct. 2025.

[24] P. LUCZYNSKI, L. GIANINAZZI, P. IFF, L. WILSON, D. DE SENSI, AND T. HOEFLER, *Near-Optimal Wafer-Scale Reduce*, June 2024, pp. 334–347.

[25] NVIDIA, *NVIDIA A100 Tensor Core GPU Datasheet*, (2022).

[26] D. PEREZ, A. THOMPSON, S. MOORE, T. OPPELSTRUP, I. SHARAPOV, K. SANTOS, A. SHARIFIAN, D. Z. KALCHEV, R. SCHREIBER, S. PAKIN, E. A. LEON, J. H. LAROS, M. JAMES, AND S. RAJAMANICKAM, *Breaking the mold: Overcoming the time constraints of molecular dynamics on general-purpose hardware*, The Journal of Chemical Physics, 162 (2025), p. 072501.

[27] C. SZEGEDY, W. LIU, Y. JIA, P. SERMANET, S. REED, D. ANGUELOV, D. ERHAN, V. VANHOUCKE, AND A. RABINOVICH, *Going Deeper with Convolutions*, Sept. 2014.

[28] R. A. VAN DE GEIJN AND J. WATTS, *SUMMA: Scalable universal matrix multiplication algorithm*, Concurrency: Practice and Experience, 9 (1997), pp. 255–274.

[29] A. VASWANI, N. SHAZEER, N. PARMAR, J. USZKOREIT, L. JONES, A. N. GOMEZ, L. KAISER, AND I. POLOSUKHIN, *Attention Is All You Need*, Aug. 2023.

[30] J. VON NEUMANN, *First draft of a report on the EDVAC*, IEEE Annals of the History of Computing, 15 (1993), pp. 27–75.

[31] J. WANG, *100× Defect Tolerance: How Cerebras Solved the Yield Problem*. https://www.cerebras.ai/blog/100x-defect-tolerance-how-cerebras-solved-the-yield-problem.

[32] Y. WANG, Y. DENG, Y. ZHENG, P. CHATTOPADHYAY, AND L. WANG, *Vision transformers for image classification: A comparative survey*, Technologies, (2025).